Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL

Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, *Senior Member, IEEE*, and Xiao Huang

(Survey Paper)

***Abstract*—Generating accurate SQL from users' natural language questions (text-to-SQL) remains a long-standing challenge due to the complexities involved in user question understanding, database schema comprehension, and SQL generation. Traditional text-to-SQL systems, which combine human engineering and deep neural networks, have made significant progress. Subsequently, pre-trained language models (PLMs) have been developed for text-to-SQL tasks, achieving promising results. However, as modern databases and user questions grow more complex, PLMs with a limited parameter size often produce incorrect SQL. This necessitates more sophisticated and tailored optimization methods, which restrict the application of PLM-based systems. Recently, large language models (LLMs) have shown significant capabilities in natural language understanding as model scale increases. Thus, integrating LLM-based solutions can bring unique opportunities, improvements, and solutions to text-to-SQL research. In this survey, we provide a comprehensive review of existing LLM-based text-to-SQL studies. Specifically, we offer a brief overview of the technical challenges and evolutionary process of text-to-SQL. Next, we introduce the datasets and metrics designed to evaluate text-to-SQL systems. Subsequently, we present a systematic analysis of recent advances in LLM-based text-to-SQL. Finally, we make a summary and discuss the remaining challenges in this field and suggest expectations for future research directions.**

***Index Terms*—Text-to-SQL, large language models, database, natural language understanding.**

I. Introduction

TEXT-TO-SQL is a long-standing task in natural language processing, aiming to convert natural language (NL) questions into database-executable SQL queries. Fig. 1 illustrates an example of a large language model-based (LLM-based) text-to-SQL system. Given a user question such as *"Could you tell me the names of the 5 leagues with the highest matches of all time and how many matches were played in the said league?"*, the LLM takes the question and its corresponding database schema as input to generate an SQL query. Executing this SQL query in the database retrieves the relevant content needed to answer the user's question. This system serves as a natural language interface to the database (NLIDB) using LLMs. Since SQL is one of the most widely used programming languages, with over half (51.52%) of professional developers employing it, but only around a third (35.29%) are systematically trained[1], the NLIDB allows non-skilled users to access structured databases like professional database engineers [2], [3] and enhances human-computer interaction [4]. Furthermore, amid the research hotspot of LLMs, text-to-SQL can potentially mitigate prevalent hallucination issues by integrating realistic content from databases to fill LLMs' knowledge gaps [5]. The

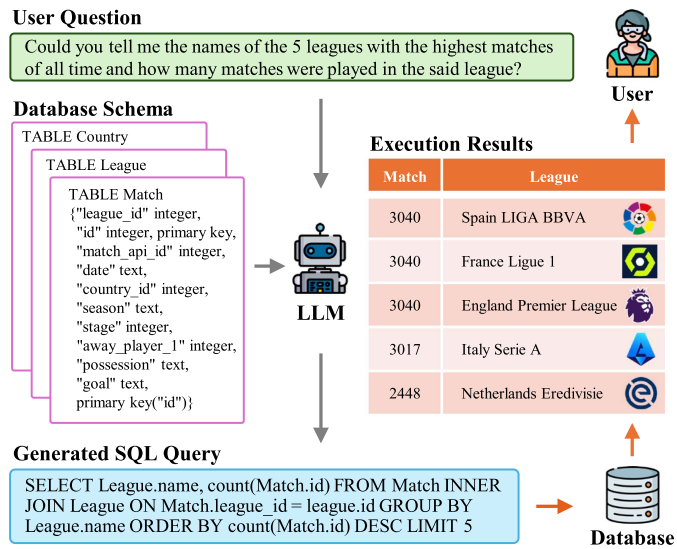

Fig. 1. An example of LLM-based text-to-SQL is selected from the BIRD [1] dataset. A user asks a question about football leagues. The LLM takes this question along with the schema of the corresponding database as input and generates an SQL query as output. The generated SQL query can be executed in the database, retrieves the content *"The 5 leagues with the highest matches"*, providing the answer to the user's question.

Received 13 July 2024; revised 12 August 2025; accepted 2 September 2025. Date of publication 12 September 2025; date of current version 10 November 2025. This work was supported in part by the Research Grants Council of the Hong Kong Special Administrative Region, China under Project PolyU under Grant 25208322, in part by NSFC under Grant 62502008, and in part by the Funding Scheme for Research and Innovation of FDCT under Grant 0021/2025/ITP1. Recommended for acceptance by Y. Shen. *(Corresponding author: Qinggang Zhang.)*

Zijin Hong, Zheng Yuan, Qinggang Zhang, Junnan Dong, and Xiao Huang are with the The Hong Kong Polytechnic University, Hong Kong, SAR, China (e-mail: zijin.hong@connect.polyu.hk; yzheng.yuan@connect.polyu.hk; qinggang.zhang@connect.polyu.hk; hanson.dong@connect.polyu.hk; xiao.huang@polyu.edu.hk).

Hao Chen is with the City University of Macau, Macau SAR, China (e-mail: sundaychenhao@gmail.com).

Feiran Huang is with the Jinan University, Guangzhou 510632, China (e-mail: huangfr@jnu.edu.cn).

All the related resources are collected for the community in our repository: https://github.com/DEEP-PolyU/Awesome-LLM-based-Text2SQL.

Digital Object Identifier 10.1109/TKDE.2025.3609486

[1]https://survey.stackoverflow.co/2023

1041-4347 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

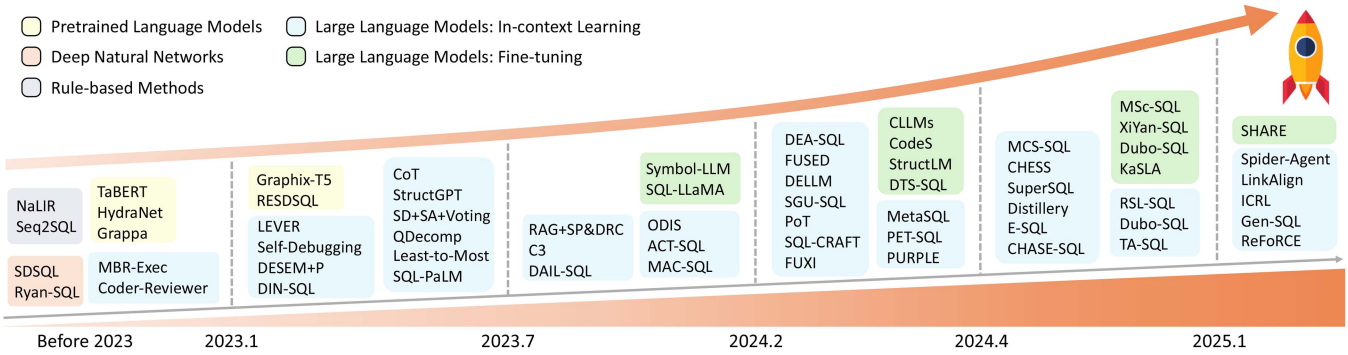

Fig. 2. A sketch of text-to-SQL research evolution outlines the advancement of implementation paradigms, each represented with different color backgrounds. Before 2023, the focus is on a selection of representative traditional studies. However, from 2023 onward, the emphasis shifts to the rapid advancements driven by LLMs, marking a significant acceleration in the field.

significant value of text-to-SQL has triggered a range of studies on its integration and optimization with LLMs [6], [7], [8]; consequently, LLM-based text-to-SQL remains a highly discussed research field in the NLP and database communities.

Previous traditional methods have made notable progress in implementing text-to-SQL. As shown in Fig. 2, these implementations have undergone a long evolutionary process. Early efforts were mostly based on well-designed rules and templates [9], specifically suitable for simple database scenarios. In recent years, due to the heavy labor costs [10] associated with rule-based methods and the growing complexity of database environments [11], [12], [13], designing a rule or template for each scenario has become increasingly difficult and impractical. The development of deep neural networks has advanced the progress of text-to-SQL [14], [15], which can automatically learn a mapping from the user question to its corresponding SQL [16], [17]. Subsequently, pre-trained language models (PLMs) with strong semantic parsing capabilities have become the new paradigm for text-to-SQL systems [18], boosting their performance to a new level [19], [20], [21]. Incremental research on PLM-based optimization, such as table content encoding [17], [18], [22] and pre-training [18], [23], has further advanced this field. Most recently, LLM-based approaches have implemented text-to-SQL through the in-context learning [7] and fine-tuning [24] paradigms, reaching state-of-the-art accuracy with well-designed frameworks and stronger comprehension capabilities compared to PLM-based methods.

The overall implementation details of LLM-based text-to-SQL can be divided into three aspects: **1. Question understanding**: The NL question is a semantic representation of the user's intention, which the corresponding generated SQL query is expected to align with; **2. Schema comprehension**: The schema provides the table and column structure of the database, and the text-to-SQL system is required to identify the target components that match the user question; **3. SQL generation**: This involves incorporating the above parsing and then predicting the correct syntax to generate executable SQL queries that can retrieve the required answer. LLMs have proven to perform a successful vanilla implementation [6], [25], benefiting from the powerful semantic parsing capacity enabled by a rich training corpus [26], [27]. Further studies on enhancing LLMs for question understanding [7], [8], schema comprehension [28], [29], and SQL generation [30] are being increasingly released.

Despite the significant progress made in text-to-SQL research, several challenges remain that hinder the development of robust and generalized text-to-SQL systems [92]. Recent related works have surveyed text-to-SQL systems using deep learning approaches and provided insights into previous deep neural network and PLM-based research [3], [27], [93], [94]. In this survey, we aim to catch up with recent advances and provide a comprehensive review of the current state-of-the-art models and approaches in LLM-based text-to-SQL. We begin by introducing the fundamental concepts and challenges associated with text-to-SQL, highlighting the importance of this task in various domains. We then delve into the evolution of the implementation paradigm for text-to-SQL systems, discussing the key advancements and breakthroughs in this field. After the overview, we provide a detailed introduction and analysis of the recent advances in text-to-SQL integrating LLMs. Specifically, the body of our survey covers a range of topics related to LLM-based text-to-SQL, including:

- *Benchmarks and Metrics:* We introduce the commonly used datasets and benchmarks for evaluating LLM-based text-to-SQL systems. We discuss their characteristics, complexity, and the challenges they pose for development and evaluation, along with the evaluation metrics.
- *Methods:* We systematically analyze different methods proposed for LLM-based text-to-SQL, including in-context learning and fine-tuning paradigms. We explore their implementation details, strengths, and specific adaptations, organized through well-designed categorization.
- *Summarization and Expectations:* We summarize the advantages and limitations of both LLM-based and traditional methods, as well as within LLM-based methods themselves. We discuss remaining challenges such as real-world robustness, computational efficiency, data privacy, and possible extensions. Additionally, we outline future research directions and opportunities for improvement.

To summarize our technical contributions, **at the time of this paper's release, we present the first LLM-based text-to-SQL survey in the community**, establishing a systematic

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

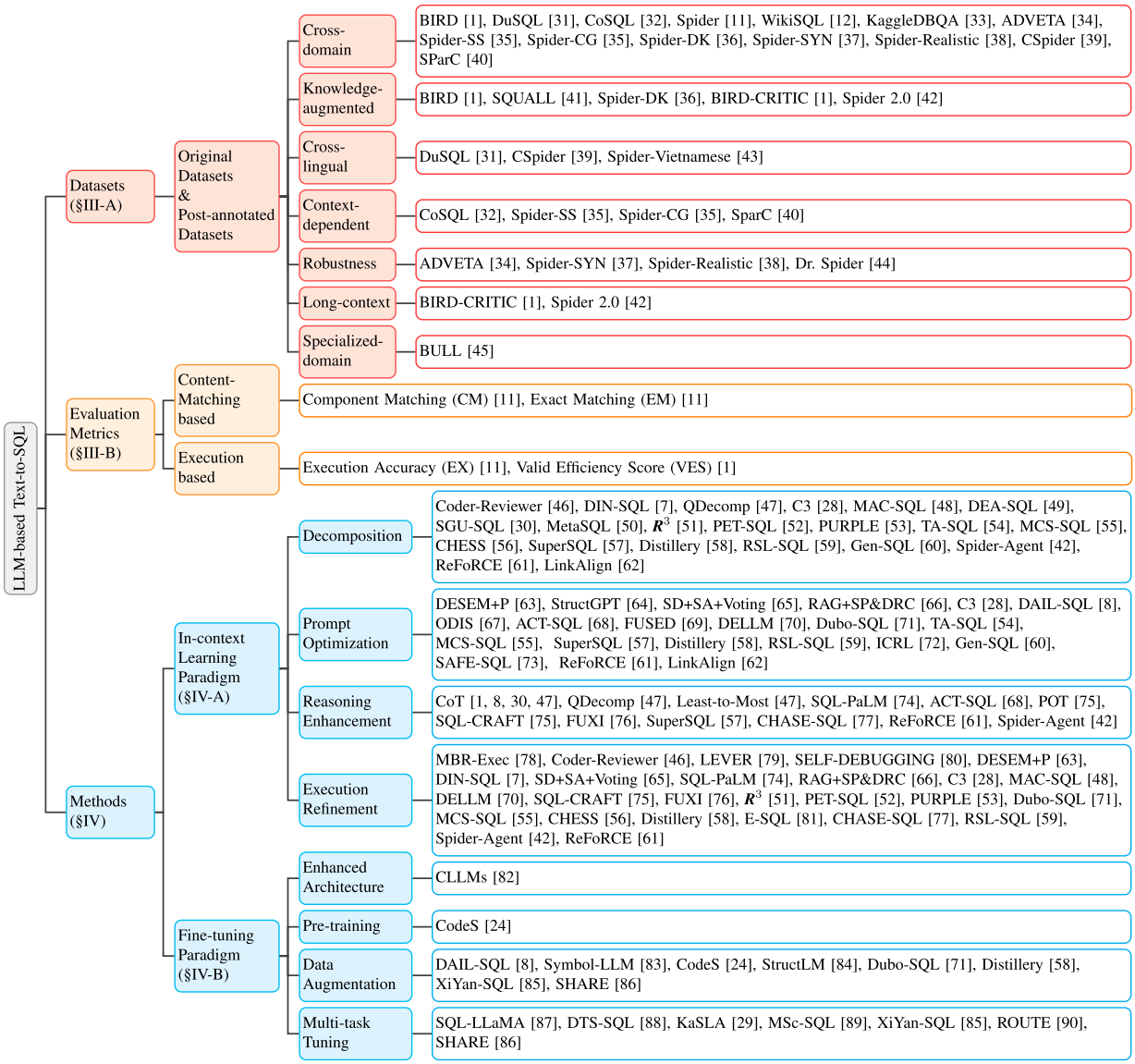

Fig. 3. Taxonomy tree of research in LLM-based text-to-SQL, taking inspiration from [91]; nodes are ordered by release time.

taxonomy for LLM-based text-to-SQL methods. **This categorization provides analysis from technical implementation perspectives**, clearly indicating their contributions to generating better SQL. Then, **we further integrate insights while analyzing the trade-offs** between LLM-based and traditional methods, as well as between in-context learning and fine-tuning paradigms for future research. Finally, **we highlight the remaining challenges** in robustness, real-world deployment, efficiency, and domain adaptation, **offering actionable directions for future research**. Fig. 3 shows a taxonomy for the survey.

II. Overview

Text-to-SQL is a task that aims to convert NL questions into SQL queries that can be executed in a relational database. It has the potential to democratize data access by allowing users to interact with databases using natural language, thus eliminating the need for specialized SQL programming knowledge [95]. This capability can greatly benefit domains such as business intelligence, customer support, and scientific research by enabling non-specialized users to easily retrieve target content from databases and facilitating more efficient data analysis.

A. Challenges in Text-to-SQL

The technical challenges are summarized as follows:

1) Linguistic Complexity and Ambiguity: Natural language questions often contain complex linguistic representations, such as nested clauses, coreferences, and ellipses, which make it challenging to map them accurately to the corresponding parts of SQL queries [38]. Additionally, natural language is inherently ambiguous, with multiple possible representations for a given user question [96], [97]. Resolving these ambiguities and understanding the intent behind the user question requires deep

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

natural language understanding and the capability to incorporate context and domain knowledge [1].

2) Schema Understanding and Representation: To generate accurate SQL queries, text-to-SQL systems must have a comprehensive understanding of the database schema, including table names, column names, and relationships between various tables. Representing and encoding schema information is challenging because the database schema is often complex and varies significantly across different domains [11].

3) Rare and Complex SQL Operations: Some SQL queries involve rare or complex operations and syntax in challenging scenarios, such as nested sub-queries, outer joins, and window functions. These operations are less frequent in the training data and pose challenges for text-to-SQL systems to generate accurately. Designing models that can generalize to a wide range of SQL operations, including rare and complex scenarios, is an essential consideration.

4) Cross-Domain Generalization: Text-to-SQL systems often struggle to generalize across various database scenarios and domains. Models trained on a specific domain may not perform well on questions from other domains due to variations in vocabulary, database schema structure, and question patterns. Developing systems that can effectively generalize to new domains with minimal domain-specific training data or fine-tuning adaptation is a significant challenge [98].

B. Evolutionary Process

The research field of text-to-SQL has witnessed significant advancements over the years in the NLP community, having evolved from rule-based methods to deep learning-based approaches and, more recently, to integrating pre-trained language models (PLMs) and large language models (LLMs), a sketch of the evolutionary process is shown in Fig. 2.

1) Rule-Based Methods: Early text-to-SQL systems relied heavily on rule-based methods [9], [10], [23], where manually crafted rules and heuristics were used to map natural language questions to SQL queries. Rule-based systems excel at generating syntactically correct SQL queries by relying on predefined templates and strict grammatical rules, ensuring adherence to SQL syntax and structure. Their deterministic nature guarantees consistent outputs for specific input patterns, minimizing variability in query generation. However, these methods struggle to interpret linguistically complex or ambiguous natural language questions, such as those involving nested clauses, coreferences, or ellipses, and often fail to map these elements to database schema components like tables or columns. Their reliance on manual rules limits generalization to unseen question patterns or schema structures, requiring frequent updates for new domains. Additionally, encoding complex schemas with multi-table relationships into usable rules is labor-intensive and prone to errors.

2) Deep Learning-Based Approaches: With the rise of deep neural networks, sequence-to-sequence models and encoder-decoder structures, such as LSTMs [99] and transformers [15], were adapted to generate SQL queries from natural language input [17], [100]. Typically, RYANSQL [17] introduced techniques like intermediate representations and sketch-based slot filling to handle complex questions and improve cross-domain generalization. Recently, researchers introduced graph neural networks (GNNs) for text-to-SQL tasks by leveraging schema dependency graphs to capture the relationships between database elements [16], [101]. Despite their flexibility, these methods often generate incorrect or incomplete SQL syntax, such as missing clauses or improper nesting, due to limited structural awareness. They also struggle with rare or complex SQL operations, like nested subqueries and window functions, because such constructs are underrepresented in training data.

3) PLM-Based Implementation: PLMs have emerged as a powerful solution for text-to-SQL, leveraging the vast amounts of linguistic knowledge and semantic understanding captured during the pre-training process. The early adoption of PLMs in text-to-SQL primarily focused on fine-tuning off-the-shelf PLMs, such as BERT [102] and RoBERTa [103], on standard text-to-SQL datasets [11], [12]. These PLMs, pre-trained on large amounts of training corpus, captured rich semantic representations and language understanding capabilities. By fine-tuning them on text-to-SQL tasks, researchers aimed to leverage the semantic and linguistic understanding of PLMs to generate accurate SQL queries [18], [100], [104]. Another line of research focuses on incorporating schema information into PLMs to improve their understanding of database structures and enable them to generate more executable SQL queries. Schema-aware PLMs are designed to capture the relationships and constraints present in the database structure [19]. While PLMs generate more accurate SQL than deep learning-based methods, they still face difficulties with complex operations like outer joins or aggregations, often producing syntactically flawed queries. Besides that, cross-domain performance drops significantly when schema structures or vocabulary differ from training data, necessitating resource-intensive fine-tuning.

4) LLM-Based Implementation: Large language models (LLMs), such as the GPT series [105], [106], [107], have gained significant attention in recent years due to their ability to generate coherent and fluent text. Researchers have begun exploring the potential of LLMs for text-to-SQL by leveraging their extensive knowledge reserves and superior generation capabilities [6], [8]. These approaches often involve prompt engineering to guide proprietary LLMs in SQL generation [108] or fine-tuning open-source LLMs on text-to-SQL datasets [8].

The integration of LLMs in text-to-SQL is still an emerging research area with significant potential for further exploration and improvement. Researchers are investigating ways to better leverage the knowledge and reasoning capabilities of LLMs, incorporate domain-specific knowledge [1], [70], and develop more efficient fine-tuning strategies [24]. As the field continues to evolve, we anticipate the development of more advanced and superior LLM-based implementations that will elevate the performance and generalization of text-to-SQL to new heights.

III. Benchmarks & Evalution

In this section, we introduce the benchmarks for text-to-SQL, encompassing well-known datasets and evaluation metrics.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE STATISTICS AND ANALYSIS OF WELL-KNOWN DATASETS OF TEXT-TO-SQL ORDERED BY RELEASE TIME. THE ORIGINAL DATASET INDICATES THAT THE DATASET IS DESIGNED WITH A CORRESPONDING DATABASE, WHILE POST-ANNOTATED DATASETS INVOLVE ANNOTATING NEW COMPONENTS WITHIN EXISTING DATASETS AND DATABASES RATHER THAN RELEASING A NEW DATABASE. FOR EACH DATASET, THE LANGUAGES UTILIZED ARE DENOTED BY THE FOLLOWING ABBREVIATIONS: EN FOR ENGLISH, ZH FOR CHINESE, AND VI FOR VIETNAMESE

Original Dataset	Release Time	#Example	#DB	#Table/DB	#Row/DB	Language	Characteristics
BIRD-CRITIC [1]	Feb-2025	600	-	-	-	EN	Knowledge-augmented, Long-context
Spider 2.0 [42]	Nov-2024	632	213	-	-	EN	Knowledge-augmented, Long-context
BULL [45]	Jan-2024	4,966	3	26	-	EN	Specialized-domain
BIRD [1]	May-2023	12,751	95	7.3	549K	EN	Cross-domain, Knowledge-augmented
KaggleDBQA [33]	Jun-2021	272	8	2.3	280K	EN	Cross-domain
DuSQL [31]	Nov-2020	23,797	200	4.1	-	EN	Cross-domain, Cross-lingual
SQUALL [41]	Oct-2020	11,468	1,679	1	-	EN	Knowledge-augmented
CoSQL [32]	Sep-2019	15,598	200	-	-	EN	Cross-domain, Context-dependent
Spider [11]	Sep-2018	10,181	200	5.1	2K	EN	Cross-domain
WikiSQL [12]	Aug-2017	80,654	26,521	1	17	EN	Cross-domain
Post-annotated Dataset	**Release Time**	**Source Dataset**	**Special Setting**			**Language**	**Characteristics**
Dr. Spider [44]	Jan-2023	Spider	Perturbations in DB, query and SQL			EN	Robustness
ADVETA [34]	Dec-2022	Spider, etc.	Adversarial table perturbation			EN	Robustness
Spider-SS&CG [35]	May-2022	Spider	Splitting example into sub-examples			EN	Context-dependent
Spider-DK [36]	Sep-2021	Spider	Adding domain knowledge			EN	Knowledge-augmented
Spider-SYN [37]	Jun-2021	Spider	Manual synonym replacement			EN	Robustness
Spider-Vietnamese [43]	Nov-2020	Spider	Vietnamese version of Spider			VI	Cross-lingual
Spider-Realistic [38]	Oct-2020	Spider	Removing column names in question			EN	Robustness
CSpider [39]	Sep-2019	Spider	Chinese version of Spider			ZH	Cross-lingual
SParC [40]	Jun-2019	Spider	Annotate conversational contents			EN	Context-dependent

A. Datasets

As shown in Table I, we categorize the datasets into "Original Datasets" and "Post-annotated Datasets" based on whether they were released with the original dataset and databases or developed by adapting existing datasets and databases with special settings. For the original datasets, we offer a detailed analysis, including the number of examples, databases, tables per database, and rows per database. For post-annotated datasets, we identify their source dataset and describe the special settings applied to them. To highlight the potential opportunities of each dataset, we annotate them according to their characteristics. The annotations are listed in the rightmost column of Table I, which we will discuss in detail below.

1) Cross-Domain Dataset: This refers to datasets where the background information of different databases comes from various domains. Since real-world text-to-SQL applications often involve databases from multiple domains, most original text-to-SQL datasets [1], [11], [12], [31], [32], [33] and post-annotated datasets [34], [35], [36], [37], [38], [39], [40] are in the cross-domain setting to fit well with the requirements of cross-domain applications.

2) Knowledge-Augmented Dataset: Interest in incorporating domain-specific knowledge into text-to-SQL tasks has increased significantly in recent years. BIRD [1] employs human database experts to annotate each text-to-SQL sample with external knowledge, categorized into numeric reasoning knowledge, domain knowledge, synonym knowledge, and value illustration. Similarly, Spider-DK [36] defines and adds five types of domain knowledge for a human-curated version of the Spider dataset [11]: SELECT columns mentioned by omission, simple inference required, synonym substitution in cell value words, one non-cell value word generates a condition, and easy to conflict with other domains. Both studies found that human-annotated knowledge significantly improves SQL generation performance for samples requiring external domain knowledge. Additionally, SQUALL [41] manually annotates alignments between the words in NL questions and the entities in SQL, providing finer-grained supervision than other datasets.

3) Context-Dependent Dataset: SParC [40] and CoSQL [32] explore context-dependent SQL generation by constructing a conversational database querying system. Unlike traditional text-to-SQL datasets that only have a single question-SQL pair for one example, SParC decomposes the question-SQL examples in the Spider dataset into multiple sub-question-SQL pairs to construct a simulated and meaningful interaction, including interrelated sub-questions that aid SQL generation, and unrelated sub-questions that enhance data diversity. CoSQL, in comparison, involves conversational interactions in natural language, simulating real-world scenarios to increase complexity and diversity. Additionally, Spider-SS&CG [35] splits the NL question in the Spider dataset [11] into multiple sub-questions and sub-SQLs, demonstrating that training on these sub-examples can improve a text-to-SQL system's generalization ability on out-of-distribution samples.

4) Robustness Dataset: Evaluating the accuracy of text-to-SQL systems with polluted or perturbed database contents (e.g., schema and tables) is crucial for assessing robustness. Spider-Realistic [38] removes explicit schema-related words from the NL questions, while Spider-SYN [37] replaces them with manually selected synonyms. ADVETA [34] introduces adversarial table perturbation (ATP), which perturbs tables by replacing original column names with misleading alternatives and inserting new columns with high semantic associations but low semantic equivalency. Based on the generation ability of large-scale pre-trained language models, Dr. Spider [44] provided 17 different perturbation strategies on databases, natural

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

language queries and SQL statements to measure the robustness of text-to-SQL systems. These perturbations lead to significant drops in accuracy, as a text-to-SQL system with low robustness may be misled by incorrect matches between tokens in NL questions and database entities.

5) Cross-Lingual Dataset: The SQL keywords, function names, table and column names are typically written in English, posing challenges for applications in other languages. CSpider [39] addresses this by translating the Spider dataset into Chinese, identifying new challenges in word segmentation and cross-lingual matching between Chinese questions and English database contents. DuSQL [31] introduces a practical text-to-SQL dataset with Chinese questions and database contents provided in both English and Chinese. Spider-Vietnamese [43] translates the Spider dataset into Vietnamese, empirically revealing several strategies to enhance text-to-SQL methods within the Vietnamese context.

6) Long-Context Dataset: In real-world database applications, numerous challenging scenarios demand complex SQL queries with significant token lengths, often exceeding 100 tokens and involving multiple intricate operations. Spider 2.0 [42] exemplifies this complexity by presenting enterprise-level text-to-SQL problems that require advanced reasoning across various SQL queries and dialects. Distinct from earlier benchmarks, Spider 2.0 contains numerous real-world tasks sourced from diverse production databases such as BigQuery and Snowflake, featuring large-scale, industrial schemas with hundreds or even thousands of columns, nested structures, and multiple schemas per instance. Each task requires navigating not only complex SQL logic, but also integrating information from project codebases, database metadata, and dialect-specific documentation, closely reflecting how data workflows operate in enterprise settings. Gold-standard queries in this benchmark average close to 150 tokens, far surpassing previous datasets in both length and complexity. Notably, all SQLs are carefully annotated and reviewed for both semantic fidelity and execution correctness, ensuring challenges are both realistic and robust. As a result, Spider 2.0 highlighted the substantial gap between present-day LLMs' capabilities and the demands of authentic, long-context database scenarios. In parallel, BIRD-CRITIC[2] [1] introduces a diagnostic dimension, comprising 600 tasks designed to assess the efficacy of LLMs in resolving real-world user issues across diverse SQL dialects. Together, these datasets push the boundaries of text-to-SQL research, emphasizing the challenges of complex, long-context SQL environments.

7) Specialized-Domain Dataset: Despite advancements in text-to-SQL research, there is still a shortage of specialized datasets tailored for domain-specific analysis. To address this gap, BULL [45] introduces a dedicated text-to-SQL benchmark for the financial sector. The dataset encompasses databases related to funds, stocks, and macroeconomic data, offering a comprehensive platform to evaluate and enhance text-to-SQL methods in financial applications, effectively tackling the financial domain's unique text-to-SQL challenges.

[2]https://bird-critic.github.io/

B. Evaluation Metrics

We introduce four widely used evaluation metrics for the text-to-SQL task as follows: Component Matching and Exact Matching, which are based on SQL content matching, and Execution Accuracy and Valid Efficiency Score, which are based on execution results.

1) Content Matching-Based Metrics: SQL content matching metrics focus on comparing the predicted SQL with the ground truth based on their structural and syntactic similarities.

- *Component Matching (CM)* [11] evaluates the performance of text-to-SQL system by measuring the exact match between predicted and ground truth SQL components—SELECT, WHERE, GROUP BY, ORDER BY, and KEYWORDS—using the F1 score. Each component is decomposed into sets of sub-components and compared for an exact match, accounting for SQL components without order constraints.
- *Exact Matching (EM)* [11] measures the percentage of examples whose predicted SQL query is identical to the ground truth SQL query. A predicted SQL is considered correct only if all its components, as described in CM, match exactly with those of the ground truth query.

2) Execution-Based Metrics: Execution result metrics assess the correctness of the generated SQL query by comparing the results obtained from executing the query on the target database with the expected results.

- *Execution Accuracy (EX)* [11] measures the correctness of a predicted SQL query by executing it in the corresponding database and comparing the executed results with the results obtained by the ground truth query.
- *Valid Efficiency Score (VES)* [1] is defined to measure the efficiency of valid SQL queries. A valid SQL query is a predicted SQL whose executed results exactly match the ground truth results. Specifically, VES evaluates both the efficiency and accuracy of predicted SQL queries. For a text dataset with N examples, VES is computed by:

$$\text{VES} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{1}(V_n, \hat{V}_n) \cdot \mathbf{R}(Y_n, \hat{Y}_n), \tag{1}$$

where $\hat{Y}_n$ and $\hat{V}_n$ are the predicted SQL query and its executed results and Y_n and V_n are the ground truth SQL query and its corresponding executed results, respectively. $\mathbb{1}(V_n, \hat{V}_n)$ is an indicator function, where:

$$\mathbb{1}(V_n, \hat{V}_n) = \begin{cases} 1, V_n = \hat{V}_n \\ 0, V_n \neq \hat{V}_n \end{cases} \tag{2}$$

Then, $\mathbf{R}(Y_n, \hat{Y}_n) = \sqrt{E(Y_n)/E(\hat{Y}_n)}$ denotes the relative execution efficiency of the predicted SQL in comparison to the ground-truth, where $E(\cdot)$ is the execution time of each SQL in the database. BIRD benchmark [1] ensures the stability of this metric by computing the average of $\mathbf{R}(Y_n, \hat{Y}_n)$ over 100 runs for each example.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

Most of the recent LLM-based text-to-SQL studies focus on BIRD and Spider-related datasets and execution-based metrics. The corresponding details can be found at Table III.

IV. Methods

The implementation of the most current LLM-based text-to-SQL methods relies on in-context learning (ICL) [109], [110], [111] and fine-tuning (FT) [112], [113] paradigms due to the release of powerful proprietary and well-architected open-source LLMs in large quantities [107], [114], [115], [116], [117], [118]. In this section, we will discuss these paradigms with detailed categorization accordingly.

A. In-Context Learning

Through extensive and widely recognized research, ICL (also called prompt engineering) has been proven to play a decisive role in the performance of LLMs across various tasks [26], [119]. Consequently, it also impacts SQL generation across different prompt styles [8], [120]. The ICL implementation of the LLM-based text-to-SQL process to generate a predicted SQL query $\hat{Y}$, can be formulated as:

$$\hat{Y} = \pi\left(I, Q, \mathcal{S} \mid \theta\right), \tag{3}$$

where I represents the instruction for the text-to-SQL task, Q represents the user question. $\mathcal{S}$ is the database schema/content, which can be decomposed as $\mathcal{S} = \langle \mathcal{C}, \mathcal{T}, \mathcal{K} \rangle$. Here, $\mathcal{C} = \{c_1, c_2, \ldots\}$ and $\mathcal{T} = \{t_1, t_2, \ldots\}$ represent the collection of various columns c and tables t, respectively. $\mathcal{K}$ is the potentially supplementary knowledge, such as foreign key relationships [68], schema linking [28], [29], and external knowledge [1], [70]. $\pi(\cdot \mid \theta)$ is a LLM π with parameter θ. In the ICL paradigm, LLM-based methods primarily utilize an off-the-shelf text-to-SQL model, meaning the parameter θ of the model is frozen, for generating the predicted SQL query.

We further specify the prompt engineering strategies in the ICL paradigm as zero-shot and few-shot prompting. Trained through massive data, LLMs have a strong overall proficiency in different downstream tasks with zero-shot and few-shot prompting [112], [121], [122], which is widely recognized and utilized in real-world applications. Formally, the overall input P_0 of zero-shot prompting for text-to-SQL can be obtained by concatenating I, $\mathcal{S}$, and Q as introduced above:

$$P_0 = I \oplus \mathcal{S} \oplus Q. \tag{4}$$

Subsequently, the overall input P_k for few-shot prompting (specifically, k-shot) can be formulated as an extension of P_0:

$$P_k = \{\mathcal{E}_1, \mathcal{E}_2, \ldots, \mathcal{E}_k\} \oplus P_0, \tag{5}$$

where k is the number of provided instances (examples) in few-shot prompting. $\mathcal{E}_i$ represents the ith few-shot instance, each of which can be decomposed as $\mathcal{E}_i = (\mathcal{S}_i, Q_i, Y_i)$, where Y_i is the gold SQL query of Q_i. The index $i \in \{1, \ldots, k\}$ also serves as the corresponding serial number for $\mathcal{S}_i$, Q_i, and Y_i. The OpenAI demonstration[3] provides an indicative example and regulation for the prompting process in LLM-based text-to-SQL [28]. The prompt designs and formats utilized by a substantial portion of the ICL methods discussed in this survey are developed based on the OpenAI regulation. Furthermore, most of these ICL methods incorporate zero-shot and few-shot prompting into their text-to-SQL frameworks.

TABLE II
A Summary of Representative Methods Used in the ICL Paradigm for LLM-Based Text-to-SQL is Provided. The Complete Table, Along With a Detailed Categorization of Existing Well-Designed Methods Denoted as $\mathbf{C}_{1:4}$, is Available in Table III. *Note That SimpleDDL is a Prompting Style That Provides Only Table and Column Names [120], Rather Than a Tailored Method

Methods	Adopted by	Applied LLMs
$\mathbf{C}_0$-Vanilla Prompting	SimpleDDL [120]*	SQLCoder
$\mathbf{C}_1$-Decomposition	DIN-SQL [7]	GPT-4
$\mathbf{C}_2$-Prompt Optimization	DAIL-SQL [8]	GPT-4
$\mathbf{C}_3$-Reasoning Enhancement	ACT-SQL [68]	GPT-4
$\mathbf{C}_4$-Execution Refinement	LEVER [79]	Codex

As the mainstream research approach in LLM-based text-to-SQL, various well-designed methods within the ICL paradigm have been adopted to enhance LLM's SQL generation. In this section, we group these existing methods into five categories, $\mathbf{C}_{0:4}$, including $\mathbf{C}_0$-Vanilla Prompting, $\mathbf{C}_1$-Decomposition, $\mathbf{C}_2$-Prompt Optimization, $\mathbf{C}_3$-Reasoning Enhancement, and $\mathbf{C}_4$-Execution Refinement. The representative methods of each category are provided in Table II. We will introduce each category accordingly and summarize their characteristics at the beginning of each corresponding paragraph.

1) $\mathbf{C}_0$-Vanilla Prompting: In our survey, we identified a category of prompting strategies that do not propose a well-designed ICL framework. These strategies are based solely on zero-shot and few-shot prompting. Despite their simplicity, they offer valuable insights and findings on the ICL paradigm in LLM-based text-to-SQL. We refer to these strategies as vanilla prompting strategies and further divide their studies into zero-shot and few-shot studies.

a) Zero-shot studies: The zero-shot-based ICL strategies primarily focus on studying the influence of prompt representation styles and the baseline performance of various LLMs for text-to-SQL [6], [25], [120]. As an empirical evaluation, [6] evaluates the baseline text-to-SQL capabilities of various early-developed LLMs [106], [123], [124] and their performance across different prompting styles. The experimental results indicate that prompt design is critical for performance. Through error analysis, this study [6] further suggests that including more database content can harm the overall accuracy of text-to-SQL. Since the emergence of ChatGPT [106] with its impressive capabilities in conversational scenarios and code generation [25], [125] evaluates its performance on Spider [11] and its variant [36], [37], [38] datasets. Under zero-shot settings, the results demonstrate that ChatGPT has promising text-to-SQL performance compared to

[3]The prompt format follows the official document provided by the OpenAI platform: https://platform.openai.com/examples/default-sql-translate

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III
WELL-DESIGNED METHODS IN THE ICL PARADIGM FOR LLM-BASED TEXT-TO-SQL ARE ORDERED BY RELEASE TIME. THE METHODS ARE GROUPED INTO FOUR CATEGORIES BASED ON THEIR IMPLEMENTATION PERSPECTIVE: **C_1-DECOMPOSITION**, **C_2-PROMPT OPTIMIZATION**, **C_3-REASONING ENHANCEMENT**, AND **C_4-EXECUTION REFINEMENT**. METHODS THAT FALL INTO MULTIPLE CATEGORIES WILL BE INTRODUCED ACCORDINGLY. THE DATASET COLUMN CONSISTS OF SIX TEXT-TO-SQL DATASETS, WHICH CAN BE CLARIFIED BY NAMES: SPIDER [11], BIRD [1], SPIDER-DK [36] SPIDER-SYN [37], SPIDER-REALISTIC [38], AND SPIDER 2.0 [42]. WE USE THE REFERENCE ID IN THE TABLE DUE TO COLUMN SPACE LIMITATIONS. *THERE ARE MULTIPLE APPLIED LLMS IN THE CORRESPONDING METHOD; THE MODEL WITH REPRESENTATIVE PERFORMANCE IS PRESENTED. †COT METHOD IS INTRODUCED IN MULTIPLE VENUES: NEURIPS'23 [1], EMNLP'23 [47], VLDB'24 [8], ARXIV'24 [30]

Methods	Applied LLMs	Dataset	Metrics	C_1	C_2	C_3	C_4	Release Time	Publication Venue
MBR-Exec [78]	Codex	[11]	EX				✓	Apr-2022	EMNLP'22
Coder-Reviewer [46]	Codex	[11]	EX	✓			✓	Nov-2022	ICML'23
LEVER [79]	Codex	[11]	EX				✓	Feb-2023	ICML'23
Self-Debugging [80]	StarCoder*	[11]	EX				✓	Apr-2023	ICLR'24
DESEM+P [63]	ChatGPT	[11, 37]	EX		✓		✓	Apr-2023	PRICAI'23
DIN-SQL [7]	GPT-4*	[1, 11, 42]	EX, EM, VES	✓			✓	Apr-2023	NeurIPS'23
CoT [1, 8, 30, 47]	GPT-4	[1, 11, 38]	EX, VES			✓		May-2023	Multiple Venues†
StructGPT [64]	ChatGPT*	[11, 37, 38]	EX		✓			May-2023	EMNLP'23
SD+SA+Voting [65]	ChatGPT*	[11, 37, 38]	EX		✓		✓	May-2023	EMNLP'23 Findings
QDecomp [47]	Codex	[11, 38]	EX	✓		✓		May-2023	EMNLP'23
Least-to-Most [47]	Codex	[11]	EX			✓		May-2023	EMNLP'23
SQL-PaLM [74]	PaLM-2	[11]	EX			✓	✓	May-2023	TMLR'24
RAG+SP&DRC [66]	ChatGPT	[11]	EX		✓		✓	Jul-2023	ICONIP'23
C3 [28]	ChatGPT	[11]	EX	✓	✓		✓	Jul-2023	arXiv'23
DAIL-SQL [8]	GPT-4*	[1, 11, 38, 42]	EX, EM, VES		✓	✓		Aug-2023	VLDB'24
ODIS [67]	Codex*	[11]	EX		✓			Oct-2023	EMNLP'23 Findings
ACT-SQL [68]	GPT-4*	[11, 37]	EX, EM		✓	✓		Oct-2023	EMNLP'23 Findings
MAC-SQL [48]	GPT-4*	[1, 11]	EX, EM, VES	✓			✓	Dec-2023	COLING'25
DEA-SQL [49]	GPT-4	[11]	EX	✓				Feb-2024	ACL'24 Findings
FUSED [69]	ChatGPT*	[11]	EX		✓			Feb-2024	EMNLP'24 Findings
DELLM [70]	GPT-4*	[1, 11]	EX, VES		✓		✓	Feb-2024	ACL'24 Findings
SGU-SQL [30]	GPT-4*	[1, 11]	EX, EM	✓				Feb-2024	ICML'25
PoT [75]	GPT-4*	[1, 11]	EX			✓		Feb-2024	arXiv'24
SQL-CRAFT [75]	GPT-4*	[1, 11]	EX			✓	✓	Feb-2024	arXiv'24
FUXI [76]	GPT-4*	[1]	EX			✓	✓	Feb-2024	EMNLP'24
MetaSQL [50]	GPT-4*	[11]	EX, EM	✓				Feb-2024	ICDE'24
$\boldsymbol{R}^3$ [51]	GPT-4*	[1, 11]	EX	✓			✓	Feb-2024	ACL'25 Workshop
PET-SQL [52]	GPT-4	[11]	EX	✓			✓	Mar-2024	arXiv'24
PURPLE [53]	GPT-4*	[11, 37, 38]	EX, EM	✓			✓	Mar-2024	ICDE'24
Dubo-SQL [71]	ChatGPT*	[1]	EX		✓		✓	Apr-2024	arXiv'24
TA-SQL [54]	GPT-4*	[1, 11, 36, 38]	EX	✓	✓			Apr-2024	ACL'24 Findings
MCS-SQL [55]	GPT-4	[1, 11]	EX, VES	✓	✓		✓	May-2024	COLING'25
CHESS [56]	GPT-4o*	[1, 11, 42]	EX, VES	✓			✓	May-2024	arXiv'24
SuperSQL [57]	GPT-4*	[1, 11]	EX, EM, VES	✓	✓	✓		Jun-2024	VLDB'24
Distillery [58]	GPT-4o-Mini*	[1]	EX	✓	✓		✓	Aug-2024	NeurIP'24 Workshop
E-SQL [81]	GPT-4o*	[1, 11]	EX, VES				✓	Sep-2024	arXiv'24
CHASE-SQL [77]	Gemini-1.5*	[1, 11]	EX, VES			✓	✓	Oct-2024	ICLR'25
RSL-SQL [59]	DeepSeek-V3*	[1, 11]	EX, VES	✓	✓		✓	Nov-2024	arXiv'24
Spider-Agent [42]	o1-preview*	[42]	EX	✓		✓	✓	Nov-2024	ICLR'25
ICRL [72]	ChatGPT	[11]	EX		✓			Jan-2025	COLING'25
Gen-SQL [60]	LLaMA-3*	[1, 11]	EX, EM, VES	✓	✓			Jan-2025	COLING'25
SAFE-SQL [73]	GPT-4o-Mini*	[11]	EX, EM		✓			Feb-2025	arXiv'25
ReFoRCE [61]	o1-preview*	[42]	EX	✓	✓	✓	✓	Feb-2025	ICLR'25 Workshop
LinkAlign [62]	DeepSeek-R1*	[1, 42]	EX, EM	✓	✓			Mar-2025	arXiv'25

state-of-the-art PLM-based systems. Aside from the proprietary models discussed above, similar evaluations indicate that open-source LLMs can also perform zero-shot text-to-SQL tasks [83], [114], [120], particularly for the code-specific models [80], [120].

Primary keys and foreign keys convey continuous knowledge across different tables, making them an effective prompting style in zero-shot scenarios. [68] studied their impact by incorporating these keys into various prompt representation styles with different database content to analyze performance outcomes. A benchmark evaluation [8] also assessed the influence of foreign keys using five distinct prompt representation styles. Each style can be viewed as a permutation and combination of instruction, rule implication, and foreign key elements. In addition to using foreign keys, this study explores zero-shot prompting combined with the rule implication *"no explanation"* to produce concise outputs. With the help of annotated external knowledge from human experts, the evaluation in the BIRD benchmark [1] significantly improves by incorporating the provided annotated oracle knowledge.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

In terms of zero-shot prompting optimization, [120] highlights the challenge of designing effective prompt templates for LLMs. Previous simple prompt constructions often lack structural uniformity, complicating the identification of specific elements in a prompt template that influence LLM performance. This challenge is addressed by investigating a more unified set of prompt templates that include various prefixes, infixes, and postfixes. To ensure fair comparability, [108] explores effective prompt construction for LLMs by studying different styles of prompt representation and highlights their findings on the crucial role of table structure, content, prompt length, and optimal representations in effective prompting.

b) Few-shot studies: The study of few-shot prompting strategies focuses on the number and selection of few-shot instances. Empirical studies have evaluated few-shot prompting for text-to-SQL across multiple datasets using various LLMs [7], [30], demonstrating solid improvements compared to zero-shot prompting. [1] provides a detailed 1-shot example to trigger the text-to-SQL model for generating accurate SQL. [75] explores the effect of the number of few-shot examples. [65] focuses on sampling strategies by examining the similarity and diversity between different demonstrations, setting random sampling as the baseline, and evaluating various strategies and their combinations for comparison. Furthermore, beyond similarity-based selection, [8] evaluates masked question similarity selection and the upper limit of similarity approaches with various numbers of few-shot examples.

A study on difficulty-level sample selection [47] compared the performance of few-shot Codex [124], using random selection and difficulty-based selection for few-shot instances on difficulty-categorized datasets [11], [38]. Three difficulty-based selection strategies are devised based on samples selected by difficulty level. [68] utilizes a hybrid strategy for selecting samples, combining static examples and similarity-based dynamic examples for few-shot prompting. In their settings, they also evaluate the impact of different input schema styles and various numbers of static and dynamic exemplars.

The impact of cross-domain examples is also being studied [67]. When incorporating in- and out-of-domain examples in different numbers, the in-domain demonstrations outperform zero-shot and out-of-domain examples, with performance improving as the number of examples increases. To explore the detailed construction of prompts, [74] compares concise and verbose prompt design strategies. The former style separates the schema, column names, primary keys, and foreign keys, while the latter organizes them as natural language descriptions.

2) $\mathbf{C}_1$-Decomposition: Decomposition-based methods reduce text-to-SQL complexity by dividing tasks or questions into manageable components, employing two main strategies:

a) Multi-module collaboration: This strategy breaks the task into specialized stages (e.g., schema linking, SQL generation, and SQL refinement) handled by distinct modules or agents. C3 [28] employs a three-module pipeline involving schema linking via clear prompting, calibration bias prompting for SQL guidance, and consistency-based voting for final selection. Similarly, MAC-SQL [48] and CHESS [56] use multi-agent frameworks where dedicated agents handle schema selection, decomposition, and refinement. Workflow-centric methods, like Distillery [58] and DEA-SQL [49], decompose the task into retrieval, generation, and correction stages. Iterative refinement and structural decomposition are also prevalent. DIN-SQL [7] integrates a four-stage pipeline, including schema linking, classification, SQL generation, and self-correction. Coder-Reviewer [46] separates generation and validation into distinct models. PET-SQL [52] refines outputs through multi-stage prompting, while MCS-SQL [55] uses execution-guided selection as the final module after multiple SQL generation. With the increasing complexity of recently proposed benchmarks, agentic frameworks like ReFoRCE [61], LinkAlign [62] and Spider-Agent [42] leveraged well-designed step-by-step action plans for LLMs to systematically reason toward the final SQL.

b) Question decomposition: Complexity is reduced by splitting user questions into intermediate sub-questions via question decomposition. QDecomp [47] uses Least-to-Most prompting, breaking complex questions into step-by-step sub-questions to facilitate SQL generation. SGU-SQL [30] employs graph-based structure linking and meta-operators for decomposing grammar-aware sub-questions. Metadata-driven approaches, such as MetaSQL [50] and TA-SQL [54], further decompose questions using metadata conditioning and task-aligned schema linking. Meanwhile, frameworks like DIN-SQL [7] and MAC-SQL [48] use question decomposition as auxiliary modules but primarily focus on module-level collaboration.

3) $\mathbf{C}_2$-Prompt Optimization: Prompt optimization methods for text-to-SQL focus on enhancing prompt engineering quality. These methods are categorized into three key strategies:

a) Advanced few-shot sampling: This strategy optimizes example selection through semantic similarity, diversity, or domain alignment. DESEM [63] and retrieval-augmented frameworks [58], [66], [71] de-semanticize questions to retrieve examples with matching intent skeletons or vector embeddings. DAIL-SQL [8] balances quality and quantity by masking domain-specific tokens and ranking examples based on the Euclidean distance between questions and SQL queries, while SAFE-SQL [73] combines skeleton masking with self-augmentation. ODIS [67] hybridizes out-of-domain and synthetic in-domain examples for robustness, and FUSED [69] enhances diversity through iterative demonstration synthesis. Dynamic selection is explored in ACT-SQL [68] and MCS-SQL [55], which adaptively retrieve examples based on defined similarity scores or masked similarity scores.

b) Schema augmentation: These methods refine schema representations to reduce redundancy and improve relevance. C3 [28] distills question-relevant schemas by removing unrelated tables or columns, while RSL-SQL [59] simplifies schemas using NL descriptions. ReFoRCE [61] employs table compression and column exploration to manage long-context schema complexity. Gen-SQL [60] bridges NL-SQL gaps by generating pseudo-schemas that align with user questions. QDecomp [47] and SD+SA+Voting [65] further augment prompts with schema-aware semantic and structural hints. For more complex databases, LinkAlign [62] introduces a schema-based question rewriting strategy, combined with a

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

retrieve-then-filter schema selection mechanism for accurate linking.

c) External knowledge integration: This group enhances prompts with domain-specific knowledge or feedback mechanisms. Knowledge-to-SQL [70] trains a DELLM to inject external knowledge about question-database relationships, while ICRL [72] integrates reinforcement learning with schema-graph-derived knowledge bases. MAC-SQL [48] and DIN-SQL [7] implicitly utilize decomposition patterns as procedural knowledge to guide generation. C3 [28] and SAFE-SQL [73] incorporate calibration bias and self-augmented feedback loops, respectively, to iteratively refine outputs.

4) $\mathbf{C}_3$-Reasoning Enhancement: Reasoning enhancement methods[4] improve LLMs' ability to handle complex SQL through structured reasoning and consistency mechanisms. These approaches focus on three crucial strategies:

a) Chain-of-thought (CoT)-based reasoning: CoT-based methods guide LLMs in generating intermediate reasoning steps before predicting SQLs. While standard CoT prompts (e.g., "Let's think step by step") show limited effectiveness in text-to-SQL [1], [8], adaptations help bridge this gap. ACT-SQL [68] automates CoT example generation by linking question slices to database columns via similarity matching. QDecomp [47] restructures SQL clauses into logical steps ordered by execution flow, reducing error propagation compared to generic CoT. CHASE-SQL [77] extends this with multi-path reasoning for candidate generation using "divide and conquer" CoT strategy.

b) Consistency-driven reasoning: These methods improve reliability through candidate diversity and agreement metrics before execution. Self-consistency [126] is employed in frameworks like C3 [28], DAIL-SQL [8], and SuperSQL [57], where majority voting selects the most frequent SQL candidate based on syntax or structural similarity. SD+SA+Voting [65] refines this by filtering candidates using schema-aware rules (e.g., valid joins, clause ordering) before voting. Unlike execution-dependent methods, these strategies prioritize logical consistency and syntactic validity over runtime feedback, making them lightweight but less robust to semantic errors.

c) Tool-augmented and agentic reasoning: Hybrid techniques combine SQL generation with external computation or tools. SQL-CRAFT [75] employs Program-of-Thoughts [127] (PoT) prompting, requiring LLMs to generate Python code alongside SQL for arithmetic-heavy queries, improving performance on datasets like BIRD [1]. FUXI [76] integrates dedicated tools for schema linking and syntax checking, dynamically invoking them during generation. These approaches differ from pure prompting strategies by explicitly bridging LLM reasoning with external symbolic systems. Also, recent frameworks such as ReFoRCE [61] and Spider-Agent [42] adopt step-by-step agentic reasoning, where LLMs act as specialized agents that iteratively plan actions to handle complex database scenarios.

5) $\mathbf{C}_4$-Execution Refinement: Execution refinement methods improve SQL accuracy by leveraging database feedback through regeneration or preferred SQL selection. These approaches are categorized into two paradigms:

a) Feedback-driven regeneration: These methods iteratively refine SQL queries using execution errors or intermediate results. Self-Debugging [80] enables LLMs to autonomously correct errors via natural language explanations of failed executions, while DESEM [63] uses error-type-specific prompts with termination criteria to prevent infinite loops. Multi-agent frameworks like MAC-SQL [48] and CHASE-SQL [77] utilize agents (e.g., Refiner, Query Fixer) to diagnose SQLite errors and regenerate SQL fixes. SQL-CRAFT [75] introduces adaptive control to balance correction depth, preventing over-refinement. Schema-focused methods like RSL-SQL [59] and DIN-SQL [7] use generic or gentle prompts to help LLMs identify schema misalignments during regeneration. Knowledge-to-SQL [70] optimizes its DELLM model through preference learning from execution feedback, while FUXI [76] integrates error feedback into tool-based reasoning. E-SQL [81] enhances questions with execution-aware candidate predictions. Agentic frameworks [42] also incorporate feedback-driven regeneration, where specialized agents iteratively analyze execution feedback to plan corrective actions to refine SQLs.

b) Execution-guided selection: This strategy leverages database execution results to select or rank candidates. ReFoRCE [61] and PET-SQL [52] prune invalid SQL queries that fail execution, then apply bias-aware voting (e.g., difficulty-weighted or efficiency-prioritized as in MCS-SQL [55]). Probabilistic verification distinguishes methods: MRC-EXEC [78] minimizes Bayes risk by comparing execution outputs to expected results, and LEVER [79] trains a verifier to predict correctness likelihoods from execution traces. Hybrid approaches like the retrieval-augmented framework [66] use execution feedback for iterative SQL revision through semantic gap analysis, bridging generation and selection.

B. Fine-Tuning

Supervised Fine-tuning (SFT) is the widely-used approach within the FT paradigm for training LLMs [27], [113]. For open-source LLMs, like LLaMA [117] and Qwen [118] series, the most straightforward way to enable the model to adapt to a specific domain is to apply SFT over collected domain-specific data. The SFT phase typically serves as an initial step in a well-designed training pipeline [128], [129], including LLM-based text-to-SQL training. Consider the prompt defined in (4) and (5) as P, and let $Y = [y_1, y_2, \ldots, y_T]$ represent the target ground-truth SQL query. The probability Pr for an LLM π to generate Y given P is formally modeled as:

$$\Pr_{\pi}(Y \mid P) = \prod_{t=1}^{T} \Pr_{\pi}(y_t \mid P, y_{<t}). \tag{6}$$

Contrary to the LLM in the ICL paradigm, the parameters of π are tuned. The SFT process minimizes the cross-entropy loss over the training dataset $\mathcal{D}$ for π:

$$\mathcal{L}_{\text{SFT}} = -\sum_{(P,Y)\in\mathcal{D}} \sum_{t=1}^{T} \log \Pr_{\pi}(y_t \mid P, y_{<t}). \tag{7}$$

[4]Unlike decomposition paradigms, these methods retain single-turn generation while enhancing intrinsic reasoning through structured prompting.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
WELL-DESIGNED METHODS USED IN THE FT PARADIGM FOR LLM-BASED TEXT-TO-SQL. THE METHODS IN EACH CATEGORY ARE ORDERED BY RELEASE TIME. *THE METHODS ARE UTILIZED IN MULTIPLE OPEN-SOURCE LLMS; WE SELECT A REPRESENTATIVE MODEL TO PRESENT

Category	Adopted by	Applied LLMs	Dataset	EX	EM	VES	Release Time	Publication Venue
Enhanced Architecture	CLLMs [82]	DeepSeek*	[11]	✓			Mar-2024	ICML'24
Pre-training	CodeS [24]	StarCoder	[1, 11]	✓		✓	Feb-2024	SIGMOD'24
Data Augmentation	DAIL-SQL [8]	LLaMA*	[11, 38]	✓	✓		Aug-2023	VLDB'24
	Symbol-LLM [83]	CodeLLaMA	[11]		✓		Nov-2023	ACL'24
	CodeS [24]	StarCoder	[1, 11, 42]	✓		✓	Feb-2024	SIGMOD'24
	StructLM [84]	CodeLLaMA	[11]		✓		Feb-2024	COLM'24
	Dubo-SQL [71]	ChatGPT	[1]	✓			Apr-2024	arXiv'24
	Distillery [58]	LLaMA-3.1*	[1]	✓			Aug-2024	NeurIPS'24 Workshop
	XiYan-SQL [85]	Qwen2.5-Coder	[1, 11]	✓			Nov-2024	arXiv'24
	SHARE [86]	LLaMA-3.1*	[1, 11, 36, 38]	✓			Jun-2025	ACL'25
Multi-task Tuning	SQL-LLaMA [48]	CodeLLaMA	[1, 11]	✓		✓	Dec-2023	COLING'25
	DTS-SQL [88]	Mistral*	[11, 37]	✓	✓		Feb-2024	EMNLP'24 Findings
	KaSLA [29]	DeepSeek-Coder*	[1, 11]	✓		✓	Oct-2024	arXiv'24
	MSc-SQL [89]	Gemma-2	[1, 11]	✓			Oct-2024	NAACL'25
	XiYan-SQL [85]	Qwen2.5-Coder	[1, 11]	✓			Nov-2024	arXiv'24
	ROUTE [90]	Qwen2.5*	[1, 11, 36–38]	✓		✓	Feb-2025	ICLR'25
	SHARE [86]	LLaMA-3.1*	[1, 11, 36, 38]	✓			Jun-2025	ACL'25

As a vanilla fine-tuning approach for LLM-based text-to-SQL, SFT is widely adopted in text-to-SQL research across various open-source LLMs [8], [24], [120]. FT paradigms are often more foundational in LLM-based text-to-SQL than ICL, which is the mainstream approach in the community. This is largely due to the comparably limited capabilities of open-source LLMs versus proprietary models in understanding complex database content. For the existing well-designed methods in the FT paradigm, we group these methods into different categories based on their training mechanisms, as shown in Table IV.

1) Enhanced Architecture: The widely-used Generative Pre-trained Transformer (GPT) framework [105] employs a decoder-only transformer architecture and conventional auto-regressive decoding for text generation. With advancements in the GPT framework, enhancements in LLM-based text-to-SQL involve modifications of the standard transformer backbone. These modifications incorporate tailored architecture to efficiently handle the structural and syntactic complexities of SQL. In traditional text-to-SQL research, this tailored architecture has effectively enhanced PLM-based performance. Techniques include schema-aware encoding [20], optimized attention [130], and targeted decoding [131], often relying on graph structures. Most recently, the speed of generating SQL queries by LLMs is notably slower than traditional PLM-based methods [19], [26], posing a challenge to building high-efficiency local NLIDBs.

As a solution, CLLMs [82] are designed to address this challenge with specialized decoding, offering speedup efficiency for SQL generation and reducing high latency.

2) Pre-Training: Pre-training is a fundamental phase of the complete training process, aimed at acquiring text generation capabilities through auto-regressive training on extensive data [132]. Code-specific LLMs (e.g., CodeLLaMA [87], StarCoder [133]) are pre-trained on code data, with diverse programming languages enabling them to generate code per user instructions [124], [134]. For SQL-specific pre-training, a key challenge is that SQL/database-related content makes up only a small part of the pre-training corpus. As a result, the open-source LLMs do not acquire a solid understanding of converting NL questions to SQL during their pre-training.

PLM-based methods propose initial attempts in pre-training paradigms, such as STAR [135] and multitask approaches [136], which improve SQL generation by leveraging SQL-guided and multitask pre-training strategies. For the most recent LLM-based text-to-SQL, the pre-training phase of the CodeS [24] model consists of three stages of incremental pre-training. Starting from a code-specific LLM [133], CodeS are incrementally pre-trained on a hybrid training corpus, including SQL-related data, NL-to-Code data, and NL-related data. The text-to-SQL understanding and performance are significantly improved.

3) Data Augmentation: During the fine-tuning process, the quality of training labels is the most straightforward factor affecting the model's performance [137]. Training with low-quality or absent labels is *"making bricks without straw."* Using high-quality or augmented data for fine-tuning consistently outperforms the meticulously designed methods relying on low-quality or raw data [27], [93]. Data-augmented fine-tuning in text-to-SQL has made substantial progress by focusing on enhancing data quality and efficiency during the SFT process.

DAIL-SQL [8] is designed as an ICL framework that uses a sampling strategy to enhance few-shot instances. Incorporating these sampled instances in the SFT process boosts the performance of open-source LLMs. Symbol-LLM [83] proposes injection and infusion stages for data-augmented instruction tuning. CodeS [24] augments the training data with bi-directional generation aided by ChatGPT. StructLM [84] trains on multiple structured knowledge tasks to improve overall capability. Dubo-SQL [71] created concise prompt templates for fine-tuning, which reduces costs and simultaneously achieves record performance. Distillery [58] employs an iterative failure-aware approach, selecting new training samples based on error analysis. XiYan-SQL [85] uses basic-syntax SFT and generation-enhanced SFT, employing hybrid augmented training data to fine-tune the SQL generator. SHARE [86] designed a rule-based error perturbation to augment schema

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

linking and final SQL inference based on the decomposed trajectories.

4) Multi-Task Tuning: Dividing a complex task into multiple tasks or utilizing multiple models is a straightforward approach to tackle complexity, as outlined in Section IV-A2 on ICL paradigms. Proprietary models in ICL-based methods boast a large number of parameters, surpassing the scale of open-source models used in fine-tuning. These models are adept at performing assigned sub-tasks effectively, facilitated by mechanisms like few-shot learning [28], [48]. To replicate this success, it is essential to assign sub-tasks strategically to open-source models, such as generating external knowledge, schema linking, and execution-based refinement, with appropriate training. This involves constructing specific datasets for fine-tuning, ultimately aiding in the final SQL generation.

SQL-LLaMA [48] is a fine-tuned LLM framework utilizing multi-agent collaboration. DTS-SQL [88] introduced a two-stage multi-task text-to-SQL fine-tuning framework, incorporating a schema-linking pre-generation task before the final SQL generation. Similarly, KaSLA [29] employed a two-stage framework using knapsack optimization as a fine-tuning strategy for the schema linking task, and SHARE [86] adopted three sequential models for SQL correction. MSc-SQL [89] trained separate models for schema linking, SQL generation, and critiquing the predicted SQL query. The ROUTE [90] framework involved the SFT process over noise filtering, data synthesis, and continued writing, validating the effectiveness of the multi-task tuning pipeline through collaboration prompting.

V. Summary

This section contrasts traditional and LLM-based approaches in addressing text-to-SQL challenges, focusing on ambiguity resolution, schema linking, complex SQL generation, and cross-domain adaptation. Additionally, we analyze differences between different categories of LLM-based approaches, while highlighting their strengths and trade-offs.

A. LLM-Based Methods versus Traditional Methods

1) Linguistic Complexity and Ambiguity: Traditional rule-based systems, like GRAPPA [23], decompose questions via syntactic templates but fail to resolve ambiguous phrasing. Neural models like RyenSQL [17] improve lexical matching but lack contextual reasoning. In contrast, LLM-based methods such as DAIL-SQL [8] leverage chain-of-thought prompting to infer dependencies stepwise, while C3-SQL [28] iteratively validates SQL drafts against paraphrased questions. While traditional methods offer lightweight disambiguation, their reliance on limited training data restricts their ability to handle unseen phrasings. LLM-based methods address this with broader pretraining, but their computational cost grows quadratically with context length, making them impractical for low-resource edge devices. A promising direction is a two-stage pipeline: use traditional methods for initial ambiguity detection, then invoke LLMs only for resolving flagged ambiguities. This hybrid approach balances efficiency and robustness.

2) Schema Understanding and Representation: Early systems rank schema elements via string similarity or schema-agnostic embeddings, truncating complex relationships. LLM-based methods enhance schema linking through in-context learning and structured attention. For example, DIN-SQL [7] serializes schema into natural language prompts with foreign-key hints, while KaSLA [29] mirrors human prioritization of schema elements, reducing the common issues of omissions and redundancies in schema linking. While rule-based methods are interpretable, their rigidity limits adaptability. LLM-based methods excel at semantic linking but lack transparency: their attention weights may prioritize incorrect columns due to spurious correlations. To mitigate this, future systems could combine symbolic rules with LLM inference: first, apply rule-based filters to exclude irrelevant columns, then use LLMs to rank the remaining candidates. A hybrid approach that combines LLMs with domain-specific rules can offer a balanced trade-off between performance and resource consumption.

3) Rare and Complex SQL Operations: Handling operations like nested sub-queries or window functions requires labor-intensive template engineering. For instance, previous rule-based methods [9] manually define 40+ SQL grammar rules to map question patterns, which could not be generalized to unseen formulations. Neural semantic parsers like PICARD [138] constrain decoding via transition-based incremental parsing, ensuring syntactically valid output by rejecting invalid tokens at each step. LLM-based methods generate rare SQLs by combining parametric knowledge from code-centric pretraining with few-shot demonstrations. For example, DAIL-SQL [8] injects prompts with examples of window functions, enabling the model to generalize to unseen formulations. Traditional methods like PICARD [138] guarantee syntactic validity via grammar-constrained decoders but cannot innovate on rare and complex SQL operations. LLM-based methods provide novel solutions but may produce invalid SQL. The most current agentic reasoning [42], [61] mitigates invalid outputs through multi-agent actions, but still suffers from error accumulation. A synergistic approach would integrate LLM's creativity with traditional validation: first, generate candidate queries with an LLM, then apply PICARD-like grammars to reject invalid tokens during beam search. This mirrors the "generate-then-verify" paradigm in program synthesis, where LLMs propose solutions and symbolic checkers enforce constraints.

4) Cross-Domain Generalization: Rule-based systems require manual template adjustments per domain, while PLM-based models like S^2 SQL [139] rely on domain-specific fine-tuning. LLM-based methods like CodeS [24] adapt to niche domains (e.g., biomedical terminology) via code-centric pretraining. Traditional domain adaptation methods require moderate fine-tuning data to align latent spaces but provide precise control over domain-specific mappings. LLM-based methods enable few-shot adaptation but risk overgeneralization. A viable solution is using LLMs as meta-optimizers: train a small adapter network for LLM using domain-specific rules, allowing the LLM to dynamically adjust its linking strategy. This combines the efficiency of rule-based adaptation with the LLM's capacity to learn cross-domain analogies.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

B. Comparison Within LLM-Based Methods

1) Comparison Within ICL Paradigm: The in-context learning (ICL) paradigm's categories exhibit complementary strengths and limitations. Decomposition ($\mathbf{C}_1$, Section IV-A2) is effective at managing complex queries through task modularization but introduces cascading error risks from multi-stage pipelines. Prompt optimization ($\mathbf{C}_2$, Section IV-A3) enhances input quality through semantic alignment but faces challenges with inherent reasoning limitations. Reasoning enhancement ($\mathbf{C}_3$, Section IV-A4) improves logical coherence through structured thinking but significantly increases computational overhead. Execution refinement ($\mathbf{C}_4$, Section IV-A5) ensures runtime validity through feedback loops but relies heavily on database accessibility. Recent methods like SuperSQL [57], Spider-Agent [42], and ReFoRCE [61] illustrate a trend toward hybrid integration-combining decomposition's structural rigor with prompt optimization's efficiency, while layering reasoning enhancement and execution consistency, which are implemented through well-designed agentic reasoning actions. This convergence suggests that future ICL methods will increasingly unify cross-category techniques into cohesive frameworks, balancing accuracy, efficiency, and robustness through systematic collaboration.

2) Comparison Within FT Paradigm: Fine-tuning (FT) approaches highlight the trade-off between specialization and generalization. Enhanced architectures such as CLLMs [82] reduce latency, while pre-training approaches like CodeS [24] offer robust SQL foundations. Consequently, both approaches are constrained by their demands on computational resources and hardware devices. Data augmentation (DAIL-SQL [8], XiYan-SQL [85]) improves data efficiency but risks overfitting to synthetic patterns. Multi-task tuning (ROUTE [90]) boosts multi-step reasoning but complicates training convergence. While current methods focus on isolated improvements, the field lacks holistic frameworks to combine architectural technical contributions. Future FT methods need choreographed pipelines that sequentially optimize pre-training objectives, data diversity strategies and task-specific adaptations. This orchestration will be crucial for open-source models to achieve the multifaceted capabilities of proprietary LLMs in text-to-SQL.

3) Comparing ICL and FT Paradigm: The paradigms present asymmetric value propositions. ICL methods leverage proprietary LLMs (e.g., GPT-4 [107]) for rapid adaptation but face constraints in controllability and domain customization. FT methods empower open-source models (e.g., CodeLLaMA [87]) with targeted SQL capabilities through fine-tuning but require substantial engineering for data curation and training optimization. Importantly, FT methods enhance foundational model competence: CodeS's [24] incremental pre-training boosts question and schema understanding, while SQL-LLaMA's [48] multi-agent tuning supports complex compositional reasoning. Nonetheless, ICL methods' prompt-based flexibility remain unmatched for addressing emerging database environments without retraining. The future landscape will likely reveal a symbiotic relationship: FT develops specialized models with general SQL grounding, while ICL offers lightweight adaptation for downstream application. This division mirrors trends in a retrieval-augmented generation, where FT builds core capabilities, and ICL manages dynamic context integration.

VI. Expectations

Despite the significant advancements made in LLM-based text-to-SQL research, there are still several challenges that need to be addressed. In this section, we discuss the remaining challenges that we expect to overcome in future work.

A. Robustness in Real-World Applications

The text-to-SQL implemented by LLMs is expected to perform generalization and robustness across complex scenarios in real-world applications. Despite recent advances having made substantial progress in robustness-specific datasets [34], [38], its performance still falls short of practical application [1]. There are still challenges that are expected to be overcome in future studies. From the user aspect, there is a phenomenon that the **user is not always a clear question proposer**, which means the user questions might not have the exact database value and also can be varied from the standard datasets, the synonyms, typos, and vague expressions could be included [37]. For instance, the models are trained on clear indicative questions with concrete expressions in the fine-tuning paradigm. Since the model has not learned the mapping of realistic questions to the corresponding database, this leads to a knowledge gap when applied to real-world scenarios [1]. As reported in the corresponding evaluations of the dataset with synonym and incomplete instruction [6], [47], the SQL queries generated by ChatGPT contain around 40% incorrect execution, which is 10% lower than the original evaluation [47]. For the evaluation in real-world enterprise tasks [42], current state-of-the-art models solve only about 21.3% of these tasks, far below their performance on earlier benchmarks [1], [11]. Simultaneously, the **fine-tuning with local text-to-SQL datasets may contain non-standardized samples and labels.** As an example, the name of the table or column is not always an accurate representation of its content, which yields an inconsistency within the training data construction and may lead to a semantic gap between the database schema and the user question. To address this challenge, aligning the LLMs with intention bias and designing the training strategy towards noisy scenarios will benefit the recent advances. At the same time, **the data size in real-world applications is relatively smaller than the research-oriented benchmark**. Since extending a large amount of the data by human annotation incurs high labor costs, designing data-augmentation methods to obtain more question-SQL pairs will support the LLM in data scarcity. Besides that, the adaptation study of fine-tuned open-source LLM to local small-size datasets can be potentially beneficial. Furthermore, **the extensions on multi-lingual [39], [43] and multi-modal scenarios** [140] should be studied in future research, which will benefit more language groups and help build more general database interfaces.

B. Computational Efficiency

The computational efficiency is determined by the inference speed and the cost of computational resources, which is worth

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

considering in both application and research work [68], [82]. With the increasing complexity of databases in up-to-date text-to-SQL benchmarks [1], [13], [42], databases will carry more information (including more tables and columns), and the token length of the database schema will correspondingly increase, raising a series of challenges. Dealing with an ultra-complex database, taking the corresponding schema as input may encounter the challenge that **the cost of calling proprietary LLMs will significantly increase, potentially exceeding the model's maximum token length**, especially with the implementation of open-source models that have shorter context lengths. Meanwhile, another obvious challenge is that most works **use the full schema as model input, which introduces significant redundancy** [48]. Providing LLMs with a precise question-related filtered schema directly from the user end to reduce cost and redundancy is a potential solution to improve computational efficiency [28]. Designing an accurate method for schema filtering remains a future direction. Although the in-context learning paradigm achieves promising accuracy, as a computational efficiency concern, the well-designed methods with the multi-stage framework or extended context **increasing the number of API calls to enhance performance has simultaneously led to a substantial rise in costs** [7]. Recent agentic reasoning approaches, relying on iterative multi-step actions, typically multiply API calls severalfold. While enabling complex task handling and improved robustness, optimizing action planning to reduce redundant interactions across diverse database scenarios remains an important direction for future research. As reported in the related study [68], a trade-off between performance and computational efficiency should be considered carefully, and designing a comparable (even better) in-context learning method with less API cost will be a practical implementation and is still under exploration. Compared to PLM-based methods, **the inference speed of LLM-based methods is observably slower** [19], [26]. Accelerating inference by shortening the input length and reducing the number of stages in implementation would be intuitive for the in-context learning paradigm. For local LLMs, from a starting point [82], more speedup strategies can be studied in enhancing the model's architecture in future exploration.

C. Data Privacy and Interpretability

As a part of the LLMs' study, LLM-based text-to-SQL also faces some general challenges present in LLM research [141], [142], [143]. Potential improvements from the text-to-SQL perspective are also expected to be seen in these challenges, thereby extensively benefiting the study of LLMs. As previously discussed in Section IV-A, the in-context learning paradigm predominates the number and performance in recent studies, with the majority of work using proprietary models for implementation [7], [8]. A straightforward challenge is proposed regarding data privacy, as **calling proprietary APIs to handle local databases with confidentiality can pose a risk of data leakage**. Using a local fine-tuning paradigm can partially address this issue. Still, the current performance of vanilla fine-tuning is not ideal [8], and advanced fine-tuning framework potentially relies on proprietary LLMs for data augmentation [24]. Based on the current status, more tailored frameworks in the local fine-tuning paradigm for text-to-SQL deserve widespread attention. Overall, the development of deep learning continually faces challenges regarding interpretability [143], [144]. As a long-standing challenge, considerable work has already been studied to address this issue [145], [146]. However, in text-to-SQL research, **the interpretability of LLM-based implementation is still not being discussed**, whether in the in-context learning or fine-tuning paradigm. The approaches with a decomposition phase explain the text-to-SQL implementation process from the perspective of step-by-step generation [7], [47]. Building on this, combining advanced study in interpretability [147], [148] to enhance text-to-SQL interpreting the model architecture from the database knowledge aspect remains a future direction.

D. Extensions

As a sub-field of LLMs and natural language understanding research, many studies in these fields have been adopted for text-to-SQL tasks, advancing its development [126], [149]. However, text-to-SQL research can also be extended to the larger scope studies of these fields at meanwhile. For instance, SQL generation is a part of code generation. The well-designed approaches in code generation also obtain promising performance in text-to-SQL [79], [80], performing generalization across various programming languages. **The potential extension of some tailored text-to-SQL frameworks to NL-to-code studies can also be discussed**. For instance, frameworks integrating execution output in NL-to-code can also achieve solid performance in SQL generation [7].

An attempt to extend execution-aware approaches in text-to-SQL with other advancing modules [28], [70] for code generation is worth discussing. From another perspective, we previously discussed that text-to-SQL can enhance LLM-based question-answering (QA) by providing factual information. The database can store relational knowledge as structural information, and **the structure-based QA can potentially benefit from text-to-SQL** (e.g., knowledge-based question-answering, KBQA [150], [151]). Construct the factual knowledge with database structure, and then incorporate the text-to-SQL system to achieve information retrieval, which can potentially assist further QA with more accurate factual knowledge [152]. More extensions of text-to-SQL studies are expected in future work.

References

[1] J. Li et al., "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 42330–42357.

[2] L. Wang et al., "Proton: Probing schema linking information from pre-trained language models for text-to-SQL parsing," in *Proc. Conf. Knowl. Discov. Data Mining*, 2022, pp. 1889–1898.

[3] B. Qin et al., "A survey on text-to-SQL parsing: Concepts, methods, and future directions," 2022, *arXiv:2208.13629*.

[4] S. Xu, S. Semnani, G. Campagna, and M. Lam, "AutoQA: From databases to QA semantic parsers with only synthetic training data," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2020, pp. 422–434.

[5] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2021, pp. 3214–3252.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

[6] N. Rajkumar, R. Li, and D. Bahdanau, "Evaluating the text-to-SQL capabilities of large language models," 2022, *arXiv:2204.00498*.

[7] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 36339–36348.

[8] D. Gao et al., "Text-to-SQL empowered by large language models: A benchmark evaluation," in *Proc. Int. Conf. Very Large Data Bases*, 2024, pp. 1132–1145.

[9] F. Li and H. V. Jagadish, "Constructing an interactive natural language interface for relational databases," in *Proc. Int. Conf. Very Large Data Bases*, 2014, pp. 73–84.

[10] T. Mahmud, K. A. Hasan, M. Ahmed, and T. H. C. Chak, "A rule based approach for NLP based query processing," in *Proc. Int. Conf. Elect. Inf. Commun. Technol. (EICT)*, 2015, pp. 78–82.

[11] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2018, pp. 3911–3921.

[12] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," 2017, *arXiv: 1709.00103*.

[13] M. Pourreza and D. Rafiei, "Evaluating cross-domain text-to-SQL models and benchmarks," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2023, pp. 1601–1611.

[14] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 3104–3112.

[15] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[16] B. Hui et al., "Improving text-to-SQL with schema dependency learning," 2021, *arXiv:2103.04399*.

[17] D. Choi, M. C. Shin, E. Kim, and D. R. Shin, "RYANSQL: Recursively applying sketch-based slot fillings for complex text-to-SQL in cross-domain databases," *Comput. Linguistics*, vol. 47, pp. 309–332, 2021.

[18] P. Yin, G. Neubig, W.-T. Yih, and S. Riedel, "TaBERT: Pretraining for joint understanding of textual and tabular data," 2020, *arXiv: 2005.08314*.

[19] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 13067–13075.

[20] J. Li et al., "Graphix-T5: Mixing pre-trained transformers with graph-aware layers for text-to-SQL parsing," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 13076–13084.

[21] D. Rai, B. Wang, Y. Zhou, and Z. Yao, "Improving generalization in language model-based text-to-SQL semantic parsing: Two simple semantic boundary-based techniques," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 150–160.

[22] Q. Lyu, K. Chakrabarti, S. Hathi, S. Kundu, J. Zhang, and Z. Chen, "Hybrid ranking network for text-to-SQL," 2020, *arXiv: 2008.04759*.

[23] T. Yu et al., "GraPPa: Grammar-augmented pre-training for table semantic parsing," in *Proc. Int. Conf. Learn. Representations*, 2021.

[24] H. Li et al., "Codes: Towards building open-source language models for text-to-SQL," in *Proc. Conf. Manage. Data*, 2024, Art. no. 127.

[25] A. Liu, X. Hu, L. Wen, and P. S. Yu, "A comprehensive evaluation of ChatGPT's zero-shot text-to-SQL capability," 2023, *arXiv:2303.13547*.

[26] J. Yang et al., "Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond," *ACM Trans. Knowl. Discov. Data*, vol. 18, 2024, Art. no. 160.

[27] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.

[28] X. Dong et al., "C3: Zero-shot text-to-SQL with ChatGPT," 2023, *arXiv:2307.07306*.

[29] Z. Yuan et al., "Knapsack optimization-based schema linking for LLM-based text-to-SQL generation," 2025, *arXiv:2502.12911*.

[30] Q. Zhang, J. Dong, H. Chen, W. Li, F. Huang, and X. Huang, "Structure guided large language model for SQL generation," in *Proc. Int. Conf. Mach. Learn.*, 2024.

[31] L. Wang et al., "DuSQL: A large-scale and pragmatic chinese text-to-SQL dataset," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2020, pp. 6923–6935.

[32] T. Yu et al., "CoSQL: A conversational text-to-SQL challenge towards cross-domain natural language interfaces to databases," in *Proc. Conf. Empir. Methods Natural Lang. Process. Int. Joint Conf. Natural Lang. Process.*, 2019, pp. 1962–1979.

[33] C.-H. Lee, O. Polozov, and M. Richardson, "KaggleDBQA: Realistic evaluation of text-to-SQL parsers," in *Proc. Assoc. Comput. Linguistics Int. Joint Conf. Natural Lang. Process.*, 2021, pp. 2261–2273.

[34] X. Pi, B. Wang, Y. Gao, J. Guo, Z. Li, and J.-G. Lou, "Towards robustness of text-to-SQL models against natural and realistic adversarial table perturbation," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 2007–2022.

[35] Y. Gan, X. Chen, Q. Huang, and M. Purver, "Measuring and improving compositional generalization in text-to-SQL via component alignment," in *Proc. Findings North Amer. Chapter Assoc. Comput. Linguistics*, 2022, pp. 831–843.

[36] Y. Gan, X. Chen, and M. Purver, "Exploring underexplored limitations of cross-domain text-to-SQL generalization," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2021, pp. 8926–8931.

[37] Y. Gan et al., "Towards robustness of text-to-SQL models against synonym substitution," in *Proc. Assoc. Comput. Linguistics Int. Joint Conf. Natural Lang. Process.*, 2021, pp. 2505–2515.

[38] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-grounded pretraining for text-to-SQL," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2021, pp. 1337–1350.

[39] Q. Min, Y. Shi, and Y. Zhang, "A pilot study for chinese SQL semantic parsing," in *Proc. Conf. Empir. Methods Natural Lang. Process. Int. Joint Conf. Natural Lang. Process.*, 2019, pp. 3650–3656.

[40] T. Yu et al., "SParC: Cross-domain semantic parsing in context," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 4511–4523.

[41] T. Shi, C. Zhao, J. Boyd-Graber, H. Daumé III, and L. Lee, "On the potential of lexico-logical alignments for semantic parsing to SQL queries," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2020, pp. 1849–1864.

[42] F. Lei et al., "Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows," in *Proc. Int. Conf. Learn. Representations*, 2025.

[43] A. Tuan Nguyen, M. H. Dao, and D. Q. Nguyen, "A pilot study of text-to-SQL semantic parsing for vietnamese," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2020, pp. 4079–4085.

[44] S. Chang et al., "Dr spider: A diagnostic evaluation benchmark towards text-to-SQL robustness," in *Proc. Int. Conf. Learn. Representations*, 2023.

[45] C. Zhang et al., "FinSQL: Model-agnostic LLMs-based text-to-SQL framework for financial analysis," in *Proc. Int. Conf. Manage. Data*, 2024, pp. 93–105.

[46] T. Zhang et al., "Coder reviewer reranking for code generation," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 41832–41846.

[47] C.-Y. Tai, Z. Chen, T. Zhang, X. Deng, and H. Sun, "Exploring chain of thought style prompting for text-to-SQL," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2023, pp. 5376–5393.

[48] B. Wang et al., "MAC-SQL: A multi-agent collaborative framework for text-to-SQL," in *Proc. Int. Conf. Comput. Linguistics*, 2024, pp. 540–557.

[49] Y. Xie et al., "Decomposition for enhancing attention: Improving LLM-based text-to-SQL through workflow paradigm," in *Findings Assoc. Comput. Linguistics*, 2024, pp. 10796–10816.

[50] Y. Fan et al., "MetaSQL: A generate-then-rank framework for natural language to SQL translation," in *Proc. Int. Conf. Data Eng.*, 2024, pp. 1765–1778.

[51] H. Xia et al., ""This is my SQL, are you with me," A consensus-based multi-agent system for text-to-SQL tasks," in *Proc. 4th Table Representation Learn. Workshop*, 2024, pp. 34–46.

[52] Z. Li et al., "PET-SQL: A prompt-enhanced two-stage text-to-SQL framework with cross-consistency," 2024, *arXiv:2403.09732*.

[53] T. Ren et al., "PURPLE: Making a large language model a better SQL writer," in *Proc. Int. Conf. Data Eng.*, 2024, pp. 15–28.

[54] G. Qu et al., "Before generation, align it! A novel and effective strategy for mitigating hallucinations in text-to-SQL generation," in *Proc. Findings Assoc. Comput. Linguistics*, 2024, pp. 5456–5471.

[55] D. Lee, C. Park, J. Kim, and H. Park, "MCS-SQL: Leveraging multiple prompts and multiple-choice selection for text-to-SQL generation," in *Proc. Int. Conf. Comput. Linguistics*, 2025, pp. 337–353.

[56] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual harnessing for efficient SQL synthesis," 2024, *arXiv:2405.16755*.

[57] B. Li, Y. Luo, C. Chai, G. Li, and N. Tang, "The dawn of natural language to SQL: Are we fully ready," in *Proc. Int. Conf. Very Large Data Bases*, 2024, pp. 3318–3331.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

[58] K. Maamari, F. Abubaker, D. Jaroslawicz, and A. Mhedhbi, "The death of schema linking? Text-to-SQL in the age of well-reasoned language models," in *Proc. 3rd Table Representation Learn. Workshop*, 2024.
[59] Z. Cao, Y. Zheng, Z. Fan, X. Zhang, and W. Chen, "RSL-SQL: Robust schema linking in text-to-SQL generation," 2024, *arXiv:2411.00073*.
[60] J. Shi et al., "Gen-SQL: Efficient text-to-SQL by bridging natural language question and database schema with pseudo-schema," in *Proc. Int. Conf. Comput. Linguistics*, 2025, pp. 3794–3807.
[61] M. Deng et al., "ReFoRCE: A text-to-SQL agent with self-refinement, format restriction, and column exploration," in *Proc. ICLR VerifAI Workshop*, 2025.
[62] Y. Wang and P. Liu, "LinkAlign: Scalable schema linking for real-world large-scale multi-database text-to-SQL," 2025, *arXiv:2503.18596*.
[63] C. Guo et al., "Prompting GPT-3.5 for text-to-SQL with desemanticization and skeleton retrieval," in *Proc. Pacific Rim Int. Conf. Artif. Intell.*, 2024, pp. 262–274.
[64] J. Jiang, K. Zhou, Z. Dong, K. Ye, X. Zhao, and J.-R. Wen, "StructGPT: A general framework for large language model to reason over structured data," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2023, pp. 9237–9251.
[65] L. Nan et al., "Enhancing text-to-SQL capabilities of large language models: A study on prompt design strategies," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2023, pp. 14935–14956.
[66] C. Guo et al., "Retrieval-augmented GPT-3.5-based text-to-SQL framework with sample-aware prompting and dynamic revision chain," in *Proc. Int. Conf. Neural Inf. Process.*, 2024, pp. 341–356.
[67] S. Chang and E. Fosler-Lussier, "Selective demonstrations for cross-domain text-to-SQL," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2023, pp. 14174–14189.
[68] H. Zhang, R. Cao, L. Chen, H. Xu, and K. Yu, "ACT-SQL: In-context learning for text-to-SQL with automatically-generated chain-of-thought," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2023, pp. 3501–3532.
[69] D. Wang, L. Dou, X. Zhang, Q. Zhu, and W. Che, "Improving demonstration diversity by human-free fusing for text-to-SQL," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2024, pp. 1193–1207.
[70] Z. Hong, Z. Yuan, H. Chen, Q. Zhang, F. Huang, and X. Huang, "Knowledge-to-SQL: Enhancing SQL generation with data expert LLM," in *Proc. Findings Assoc. Comput. Linguistics*, 2024, pp. 10997–11008.
[71] D. G. Thorpe, A. J. Duberstein, and I. A. Kinsey, "Dubo-SQL: Diverse retrieval-augmented generation and fine tuning for text-to-SQL," 2024, *arXiv:2404.12560*.
[72] R. Toteja, A. Sarkar, and P. M. Comar, "In-context reinforcement learning with retrieval-augmented generation for text-to-SQL," in *Proc. Int. Conf. Comput. Linguistics*, 2025, pp. 10390–10397.
[73] J. Lee, I. Baek, B. Kim, and H. Lee, "SAFE-SQL: Self-augmented in-context learning with fine-grained example selection for text-to-SQL," 2025, *arXiv:2502.11438*.
[74] R. Sun et al., "SQL-PaLM: Improved large language model adaptation for text-to-SQL," *Trans. Mach. Learn. Res.*, vol. 2024, 2024.
[75] H. Xia et al., "SQL-CRAFT: Text-to-SQL through interactive refinement and enhanced reasoning," 2024, *arXiv:2402.14851*.
[76] Y. Gu et al., "Middleware for LLMs: Tools are instrumental for language agents in complex environments," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2024, pp. 7646–7663.
[77] M. Pourreza et al., "CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in text-to-SQL," in *Proc. Int. Conf. Learn. Representations*, 2025.
[78] F. Shi, D. Fried, M. Ghazvininejad, L. Zettlemoyer, and S. I. Wang, "Natural language to code translation with execution," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2022, pp. 3533–3546.
[79] A. Ni et al., "LEVER: Learning to verify language-to-code generation with execution," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 26106–26128.
[80] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. Int. Conf. Learn. Representations*, 2024.
[81] H. A. Caferoğlu and Ö. Ulusoy, "E-SQL: Direct schema linking via question enrichment in text-to-SQL," 2024, *arXiv:2409.16751*.
[82] S. Kou, L. Hu, Z. He, Z. Deng, and H. Zhang, "CLLMs: Consistency large language models," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 25426–25440.
[83] F. Xu et al., "Symbol-LLM: Towards foundational symbol-centric interface for large language models," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 13091–13116.
[84] A. Zhuang et al., "StructLM: Towards building generalist models for structured knowledge grounding," in *Proc. Conf. Lang. Model.*, 2024.
[85] Y. Gao et al., "XiYan-SQL: A multi-generator ensemble framework for text-to-SQL," 2024, *arXiv:2411.08599*.
[86] G. Qu et al., "SHARE: An SLM-based hierarchical action CorREction assistant for text-to-SQL," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 11268–11292.
[87] B. Roziere et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.
[88] M. Pourreza and D. Rafiei, "DTS-SQL: Decomposed text-to-SQL with small large language models," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2024, pp. 8212–8220.
[89] S. K. Gorti et al., "MSc-SQL: Multi-sample critiquing small language models for text-to-SQL translation," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics*, 2024, pp. 2145–2160.
[90] Y. Qin et al., "ROUTE: Robust multitask tuning and collaboration for text-to-SQL," in *Proc. Int. Conf. Learn. Representations*, 2025.
[91] D. Xu et al., "Large language models for generative information extraction: A survey," 2023, *arXiv:2312.17617*.
[92] G. Katsogiannis-Meimarakis and G. Koutrika, "A survey on deep learning approaches for text-to-SQL," *J. Very Large Data Bases*, vol. 32, pp. 905–936, 2023.
[93] N. Deng, Y. Chen, and Y. Zhang, "Recent advances in text-to-SQL: A survey of what we have and what we expect," in *Proc. Int. Conf. Comput. Linguistics*, 2022, pp. 2166–2187.
[94] W. Zhang et al., "Natural language interfaces for tabular data querying and visualization: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6699–6718, Nov. 2024.
[95] P. Ma and S. Wang, "MT-teql: Evaluating and augmenting neural NLIDB on real-world linguistic and schema variations," in *Proc. Int. Conf. Very Large Data Bases*, 2021, pp. 569–582.
[96] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000 questions for machine comprehension of text," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2016, pp. 2383–2392.
[97] P. Rajpurkar, R. Jia, and P. Liang, "Know what you don't know: Unanswerable questions for SQuAD," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 784–789.
[98] H. Yang, Y. Zhang, J. Xu, H. Lu, P.-A. Heng, and W. Lam, "Unveiling the generalization power of fine-tuned large language models," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2024, pp. 884–899.
[99] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, pp. 1735–1780, 1997.
[100] J. Guo et al., "Towards complex text-to-SQL in cross-domain database with intermediate representation," 2019, *arXiv: 1905.08205*.
[101] X. Xu, C. Liu, and D. Song, "SQLNet: Generating structured queries from natural language without reinforcement learning," 2017, *arXiv: 1711.04436*.
[102] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
[103] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv: 1907.11692*.
[104] L. Dou et al., "Towards knowledge-intensive text-to-SQL semantic parsing with formulaic knowledge," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2022, pp. 5240–5253.
[105] A. Radford et al., "Improving language understanding by generative pre-training," *OpenAI Blog*, 2018.
[106] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
[107] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[108] S. Chang and E. Fosler-Lussier, "How to prompt LLMs for text-to-SQL: A study in zero-shot, single-domain, and cross-domain settings," in *Proc. NeurIPS 2nd Table Representation Learn. Workshop*, 2023.
[109] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," 2024, *arXiv:2402.07927*.
[110] J. Wang et al., "Prompt engineering for healthcare: Methodologies and applications," 2023, *arXiv:2304.14670*.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

[111] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering in large language models: A comprehensive review," 2023, *arXiv:2310.14735*.
[112] J. Wei et al., "Finetuned language models are zero-shot learners," 2021, *arXiv:2109.01652*.
[113] Y. Zheng, R. Zhang, J. Zhang, Y. Ye, and Z. Luo, "LlamaFactory: Unified efficient fine-tuning of 100 language models," 2024, *arXiv:2403.13372*.
[114] S. Xue et al., "DB-GPT: Empowering database interactions with private large language models," 2024, *arXiv:2312.17449*.
[115] T. Wang et al., "DBCopilot: Scaling natural language querying to massive databases," 2023, *arXiv:2312.03463*.
[116] H. Touvron et al., "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
[117] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[118] J. Bai et al., "Qwen technical report," 2023, *arXiv:2309.16609*.
[119] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, 2019.
[120] B. Zhang et al., "Benchmarking the text-to-SQL capability of large language models: A comprehensive evaluation," 2024, *arXiv:2403.02951*.
[121] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in *Proc. Conf. Hum. Factors Comput. Syst.*, 2021, Art. no. 314.
[122] X. Ye and G. Durrett, "The unreliability of explanations in few-shot prompting for textual reasoning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 30378–30392.
[123] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, pp. 5485–5551, 2020.
[124] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
[125] P. P. Ray, "ChatGPT: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope," *Internet Things Cyber-Physical Syst.*, vol. 3, pp. 121–154, 2023.
[126] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. Learn. Representations*, 2023.
[127] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *Trans. Mach. Learn. Res.*, vol. 2023, 2023.
[128] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 53728–53741.
[129] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 27730–27744.
[130] Z. Gong and Y. Sun, "Graph reasoning enhanced language models for text-to-SQL," in *Proc. Int. Conf. Inf. Retrieval*, 2024, pp. 2447–2451.
[131] R. Cao et al., "A heterogeneous graph to abstract syntax tree framework for text-to-SQL," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 13796–13813, Nov. 2023.
[132] D. Erhan, A. Courville, Y. Bengio, and P. Vincent, "Why does unsupervised pre-training help deep learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2010, pp. 201–208.
[133] R. Li et al., "StarCoder: May the source be with you," *Trans. Mach. Learn. Res.*, vol. 2023, 2023.
[134] Y. Wang et al., "Aligning large language models with human: A survey," 2023, *arXiv:2307.12966*.
[135] Z. Cai et al., "STAR: SQL guided pre-training for context-dependent text-to-SQL parsing," in *Proc. Findings Empir. Methods Natural Lang. Process.*, 2022, pp. 1235–1247.
[136] R. Giaquinto et al., "Multitask pretraining with structured knowledge for text-to-SQL generation," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 11067–11083.
[137] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, "Learning from noisy labels with deep neural networks: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 8135–8153, Nov. 2023.
[138] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2021, pp. 9895–9901.
[139] B. Hui et al., "Injecting syntax to question-schema interaction graph encoder for text-to-SQL parsers," in *Proc. Findings Assoc. Comput. Linguistics*, 2022, pp. 1254–1262.
[140] Y. Song, R. C.-W. Wong, and X. Zhao, "Speech-to-SQL: Toward speech-driven SQL query generation from natural language question," *J. Very Large Data Bases*, vol. 33, pp. 1179–1201, 2024.
[141] B. Yan et al., "On protecting the data privacy of large language models (LLMs): A survey," 2024, *arXiv:2403.05156*.
[142] Y. Zhang et al., "Siren's song in the AI ocean: A survey on hallucination in large language models," 2023, *arXiv:2309.01219*.
[143] C. Singh, J. P. Inala, M. Galley, R. Caruana, and J. Gao, "Rethinking interpretability in the era of large language models," 2024, *arXiv:2402.01761*.
[144] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei, "Knowledge neurons in pretrained transformers," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 8493–8502.
[145] N. Zhang et al., "A comprehensive study of knowledge editing for large language models," 2024, *arXiv:2401.01286*.
[146] K. Meng, A. S. Sharma, A. J. Andonian, Y. Belinkov, and D. Bau, "Mass-editing memory in a transformer," in *Proc. Int. Conf. Learn. Representations*, 2023.
[147] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 17359–17372.
[148] C. Zheng et al., "Can we edit factual knowledge by in-context learning," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2023, pp. 4862–4876.
[149] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
[150] H. Luo et al., "ChatKBQA: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models," 2023, *arXiv:2310.08975*.
[151] Z. Li et al., "FlexKBQA: A flexible LLM-powered framework for few-shot knowledge base question answering," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 18608–18616.
[152] G. Xiong, J. Bao, and W. Zhao, "Interactive-KBQA: Multi-turn interactions for knowledge base question answering with large language models," 2024, *arXiv:2402.15131*.

Zijin Hong received the bachelor's degree in applied mathematics with mathematics from Jinan University and the University of Birmingham. He is a first-year research post-graduate student with the DEEP Lab, Department of Computing, The Hong Kong Polytechnic University, Hong Kong SAR, under the supervision of Dr. Xiao Huang. His recent research interests include large language models and their applications, machine learning, information extraction, code generation, and especially text-to-SQL.

Zheng Yuan received the bachelor's degree in computer science and technology from Shenzhen University, in 2021. He is a second-year PhD degree with the Department of Computing, The Hong Kong Polytechnic University. He is currently a group member in the DEEP Lab supervised by Dr. Xiao Huang with the Hong Kong Polytechnic University. His research interests focus on large language models and their applications in data-driven scenarios, including retrieval-augmented generation, text-to-SQL, and recommender systems.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.

Qinggang Zhang received the bachelor degree of engineering in computer science from Northwestern Polytechnical University. He is currently working toward the fourth-year PhD degree with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong SAR. He is currently a group member in DEEP Lab supervised by Dr. Xiao Huang with The Hong Kong Polytechnic University. His research interests include knowledge graphs, large language models, and retrieval-augmented generation. He has published 6 first-author papers and more than 10 coauthor papers in top conferences and journals while serving as a reviewer for *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *IEEE Transactions on Knowledge and Data Engineering*, NeurIPS, ICML, ICLR, KDD, and ACL.

Hao Chen received the PhD degree from Beihang University, China, in 2022. He is an assistant professor with the Faculty of Data Science, City University of Macau. Before that, he worked as a postdoctoral research fellow with The Hong Kong Polytechnic University. He has published more than 40 papers, such as *IEEE Transactions on Knowledge and Data Engineering*, NeurIPS, ACL, SIGKDD, WWW, and SIGIR, etc. He received the Best Paper Award Honorable Mention with SIGIR 2023. His research interests include recommender systems, graph neural networks, and large language models.

Junnan Dong is currently working toward the final-year PhD degree with the Department of Computing, Hong Kong Polytechnic University. He focuses on large language models, knowledge graphs and graph retrieval augmented generation for advanced and knowledgeable domain-specific reasoning. He has published more than ten papers on top-tier conferences, including NeurIPS, ACL, WWW, KDD, SIGIR and WSDM. He has been continuously serving as a program committee member for NeurIPS, ICML, ICLR, ACL and WWW.

Feiran Huang (Senior Member, IEEE) received the PhD degree from the School of Computer Science and Engineering, Beihang University, Beijing, China, in 2018. He is currently a professor with the College of Information Science and Technology, Jinan University. He has more than 100 publications appearing in top conferences and journals, such as SIGIR, KDD, WWW, MM, *IEEE Transactions on Image Processing*, *International Journal of Computer Vision*, and *IEEE Transactions on Knowledge and Data Engineering*. He received 5 best paper awards, such as the Best Paper Award Honourable Mention in SIGIR 2023 and Best Paper Award Runner Up in PAKDD 2023. He holds more than 50 US, Chinese, and international granted patents. He is an editorial board member of *IEEE Transactions on Affective Computing*, *ACM Transactions on Recommender Systems*, etc. His research interests include recommender systems, social networks, sentiment analysis, and large language models.

Xiao Huang received the BS degree in engineering from Shanghai Jiao Tong University, in 2012, the MS degree in electrical engineering from the Illinois Institute of Technology, in 2015, and the PhD degree in computer engineering from Texas A&M University, in 2020. He is an assistant professor with the Department of Computing, The Hong Kong Polytechnic University. His scholarly contributions are highly regarded within the academic community, amassing more than 4,500 citations. He received the Best Paper Award Honorable Mention with SIGIR 2023. He has successfully led or completed seven research projects as principal investigator.

Authorized licensed use limited to: South-Central Minzu University. Downloaded on April 22,2026 at 08:10:21 UTC from IEEE Xplore. Restrictions apply.